

Tribhuvan University

Faculty of Humanities and Social Sciences

A PROJECT REPORT ON

QuickShip: A Smart Courier Management System

Submitted to

Department of Computer Application

Xavier International College

In partial fulfillment of the requirements for Bachelors in Computer Application

Submitted by

Bibusa Shrestha (6-2-546-25-2021)

Soichchha Khadka (6-2-546-40-2021)

Under the Supervision of

Mr. Roshan Jha



Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

SUPERVISOR`S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **"Soichchha Khadka and Bibusa Shrestha"** entitled "**QuickShip: A Smart Courier Management System**" is partial fulfillment of the requirement for the degree of Bachelor of Computer Application is recommended for the final evaluation.

Mr. Roshan Jha

SUPERVISOR



Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

LETTER OF APPROVAL

This is to certify that the project prepared by **"Soichchha Khadka and Bibusa Shrestha"** entitled "**QuickShip: A Smart Courier Management System**" in partial fulfillment of the requirement for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

..... Mr. Roshan Jha Supervisor Xavier International College Boudha, Kathmandu	..... Mr. Rajesh Nepal Director Xavier International College Boudha, Kathmandu
..... Internal Examiner	..... External Examiner

ABSTRACT

QuickShip: A Smart Courier Management System is a full-stack web application designed to streamline courier booking, **parcel tracking, staff assignment**, delivery management, feedback, and payment processing. It provides a centralized platform for customers, delivery staff, and administrators, improving operational efficiency, transparency, and delivery coordination.

Customers can book parcels, view delivery costs, track parcels using tracking IDs or QR codes, access booking history and invoices, make payments through eSewa or Cash on Delivery, and submit ratings and reviews. Administrators can manage users and staff, monitor parcel and delivery operations, review activity logs, and assign delivery staff using system recommendations. Delivery staff can manage assigned parcels, update delivery statuses, confirm successful deliveries, and record failed delivery attempts with rescheduling options.

The system incorporates **Haversine-based distance calculation**, a rule-based **Weighted Staff Recommendation algorithm**, and a **Random Forest Regressor** for predicting delivery time from historical delivery records. The machine learning model is evaluated using k-fold cross-validation with Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). A rule-based Virtual Help Assistant is also included to respond to common courier-related queries.

QuickShip is developed using React.js with Vite, Tailwind CSS, Node.js, Express.js, MySQL, and Sequelize, following a **three-tier architecture**. The system integrates eSewa payment processing, Cash on Delivery, QR-based parcel tracking, and email notifications. However, the current implementation does not include live GPS tracking, route optimization, or fully implemented password-reset functionality.

Keywords: *Courier Management System, Parcel Tracking, Random Forest, Machine Learning, Haversine Algorithm, React.js, Node.js, MySQL, Three-Tier Architecture*

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the completion of our project, **"QuickShip: A Smart Courier Management System."**

First, we would like to express our deepest appreciation to our supervisor, **Mr. Roshan Jha**, for his valuable guidance, continuous support, constructive suggestions, and encouragement throughout the development of this project. His supervision helped us improve our understanding and complete the project successfully.

We are grateful to **Xavier International College, Boudha, Kathmandu**, and the **Department of Computer Application** for providing us with the opportunity, resources, and academic environment necessary to carry out this project as part of the requirements for the Bachelor of Computer Application degree.

We would also like to thank our teachers, friends, and colleagues for their valuable suggestions, encouragement, and support during the development and documentation of the project.

Finally, we are deeply thankful to our family members for their continuous encouragement, patience, and support throughout our academic journey and during the completion of this project.

Table of Contents

SUPERVISOR`S RECOMMENDATION ..... **i**
LETTER OF APPROVAL ..... **ii**
ABSTRACT ..... **iii**
ACKNOWLEDGEMENT ..... **iv**
LIST OF ABBREVIATIONS ..... **vi**
LIST OF FIGURES ..... **vii**
LIST OF TABLES ..... **viii**
CHAPTER 1: INTRODUCTION **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitations 2
1.5 Development Methodology 4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW **6**
2.1 Background Study 6
2.2 Literature Review 7
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **10**
3.1 System Analysis 10
3.1.1 Requirement Analysis 10
3.1.2 Feasibility Analysis 13
3.1.3 System Object Modelling Using Class and Object Diagrams 14
..... 15
3.1.4 Dynamic Modelling Using State and Sequence Diagrams 16
3.1.5 Process Modelling Using Activity Diagrams 19
3.2 System Design 21
3.2.1 Mockups and High-Level Designing 21
3.2.2 Refinement of Class, Object, State, Sequence, and Activity Diagrams 24
3.2.3 Component Diagrams 24
3.2.4 Deployment Diagrams 25
3.3 Algorithm Details 26
CHAPTER 4: IMPLEMENTATION AND TESTING **31**

4.1 Implementation 31
4.2 Testing 37
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS 43
5.1 Lesson Learnt / Outcome 43
5.2 Conclusion 43
5.3 Future Recommendations 44
REFERENCES 46
APPENDICES 47

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CMS	Courier Management System
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
ETA	Estimated Time of Arrival
JS	JavaScript
JWT	JSON Web Token
MAE	Mean Absolute Error
ML	Machine Learning
ORM	Object-Relational Mapping
QR	Quick Response (code)
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
UX	User Experience

LIST OF FIGURES

Figure 1.1: Waterfall Development Methodology 4

Figure 3.1: Use Case Diagram of QuickShip 12

Figure 1.2: GANTT Chart of QuickShip Project Development 14

Figure 3.3: Class Diagram of QuickShip 15

Figure 3.4: Object Diagram of QuickShip 16

Figure 3.5: State Diagram of QuickShip Parcel Delivery Lifecycle 17

Figure 3.6: Sequence Diagram of the QuickShip 18

Figure 3.7: Activity Diagram of QuickShip Parcel Booking 20

Figure 3.8: Landing Page of QuickShip 22

Figure 3.9: Client Dashboard of QuickShip 22

Figure 3.10: Parcel Booking Page of QuickShip 23

Figure 3.11: Admin Dashboard of QuickShip 23

Figure 3.12: Staff Dashboard of QuickShip 23

Figure 3.13: Backend Component Diagram of QuickShip 25

Figure 3.14: Deployment Diagram of QuickShip 26

Figure 4.1: User Management Module (Auth Middleware) 33

Figure 4.2: Parcel Booking Module (Cost & ML ETA) 33

Figure 4.3: Weighted Staff Assignment Module 34

Figure 4.4: Delivery Status Update & Confirmation Module 35

Figure 4.5: Payment Module (eSewa Integration) 35

Figure 4.6: Parcel Tracking & QR Module 36

Figure 4.7: Random Forest ML Prediction Pipeline 37

LIST OF TABLES

TABLE 3.1: WEIGHTED STAFF RECOMMENDATION — SCORING FACTO

....28

TABLE 3.2: FEATURES AND TARGET 29

TABLE 4.1: TEST CASE FOR USER REGISTRATION / LOGIN 38

TABLE 4.2: TEST CASE FOR BOOK PARCEL 38

TABLE 4.3: TEST CASE FOR PARCEL STATUS TRACKING 39

TABLE 4.4: TEST CASE FOR BOOKING HISTORY 39

TABLE 4.5: TEST CASE FOR PAYMENT (ESEWA / COD) 40

TABLE 4.6: TEST CASE FOR STAFF DASHBOARD 40

TABLE 4.7: TEST CASE FOR ADMIN PANEL 41

TABLE 4.8: TEST CASE FOR ML DELIVERY PREDICTION 42

CHAPTER 1: INTRODUCTION

1.1Introduction

Courier services are essential in modern society, enabling the transportation of goods, documents, and parcels efficiently. With the rapid growth of e-commerce and online businesses, the demand for reliable and transparent courier services has increased significantly. Many companies, however, still rely on manual or semi-automated systems for booking, tracking, and delivery management, leading to delays, misplaced parcels, and inconsistent customer communication.

QuickShip: A Smart Courier Management System is a full-stack, web-based solution designed to manage the courier process from booking through delivery. It integrates clients, administrators, and delivery personnel into a single platform for parcel booking, status tracking, staff assignment, delivery-exception handling, payment, feedback, and reporting. The system uses eSewa for online payments and Cash on Delivery as an alternative. It also includes a Random Forest Regressor for delivery-time prediction, a rule-based Weighted Staff Recommendation algorithm for staff selection, and a rule-based Virtual Help Assistant.

By combining courier management, digital payments, and an evaluated predictive model, QuickShip improves operational efficiency, reduces errors, and provides a smooth end-to-end experience for both clients and service providers. This project highlights the role of modern web technology and applied machine learning in modernizing courier operations and supporting the digital transformation of logistics services.

1.2 Problem Statement

Traditional courier management processes can suffer from poor record management, inefficient staff assignment, weak visibility into delivery progress, and inconsistent communication between staff and customers. Manual handling of courier bookings and delivery records increases the chances of errors, misplaced information, and delayed updates.

Customers may face difficulty tracking parcel status or estimating delivery time, while administrators may struggle to assign staff fairly, monitor delivery progress, and review

parcel-related actions. Failed delivery attempts such as receiver unavailability, wrong addresses, or unreachable phones may also be poorly documented, making rescheduling difficult to manage consistently. These issues create a need for a centralized system that improves transparency and accountability while providing a data-based delivery-time estimate.

1.3 Objectives

The objectives of QuickShip: A Smart Courier Management System are:

- To develop a full-stack, web-based courier management system using React.js, Node.js/Express, and MySQL that covers the booking, payment, tracking, assignment, delivery, feedback, and reporting workflow.
- To implement the Haversine algorithm for calculating the great-circle distance between sender and receiver locations, in order to compute delivery cost transparently and to feed a delivery-time estimate.
- To design and train a Random Forest Regressor using available historical delivered-parcel records to predict delivery time in hours, and evaluate its performance using k-fold cross-validation with MAE, RMSE, and R^2 where applicable.
- To design a transparent, rule-based Weighted Staff Recommendation algorithm that recommends suitable delivery staff based on current workload, proximity, availability, and historical delivery success rate, while allowing administrative assignment or reassignment when required.
- To provide role-based access control for client, staff, and admin users, JWT-based authentication with bcrypt password hashing, QR-code tracking, activity logging for selected parcel operations, and a rule-based Virtual Help Assistant.

1.4 Scope and Limitations

The QuickShip system is designed to provide an efficient and transparent solution for managing core courier operations. The scope covers parcel booking, cost calculation, status tracking, staff assignment, delivery confirmation, failed-attempt recording and rescheduling, payment, feedback, analytics, email notifications, and delivery-time

prediction. The system does not claim to provide live GPS tracking or turn-by-turn route optimization.

1.4.1 Scope

- Online parcel booking, transparent cost breakdown, delivery-time estimate, and QR-code generation
- Parcel status tracking by tracking ID or camera-based QR scan, with a chronological tracking timeline
- Role-based access for admin, staff, and client
- Weighted, rule-based staff recommendation with options for administrative manual assignment and reassignment.
- Delivery status management, delivery confirmation, failed-attempt recording, and optional rescheduling
- Integration of the eSewa payment gateway and cash-on-delivery
- Customer feedback and star ratings
- Random Forest-based delivery-time prediction using historical delivery records, k-fold cross-validation, and a rule-based fallback estimate when sufficient training data is unavailable
- Admin analytics, delivery-performance reporting, feedback management, and parcel activity logging

1.4.2 Limitations

- The system is fully web-based and requires a stable internet connection for normal operation and external services such as eSewa and email.
- Live GPS tracking of delivery staff and automated turn-by-turn route optimization are not implemented. The staff proximity factor uses the destination of the most recent active delivery as an approximation.
- The Random Forest model requires at least 15 usable delivered-parcel records with delivery times and coordinates before training. Until then, the system uses a clearly labeled rule-based fallback estimate.

- Email notifications are implemented for booking confirmation, staff assignment, and delivery completion when SMTP settings are configured. SMS notifications are not implemented.

1.5 Development Methodology

The system is developed using the Waterfall Model, which follows a sequential and structured approach. The phases include requirement analysis, system design, implementation, testing, and maintenance. This methodology ensures proper documentation and systematic progress throughout the development lifecycle.

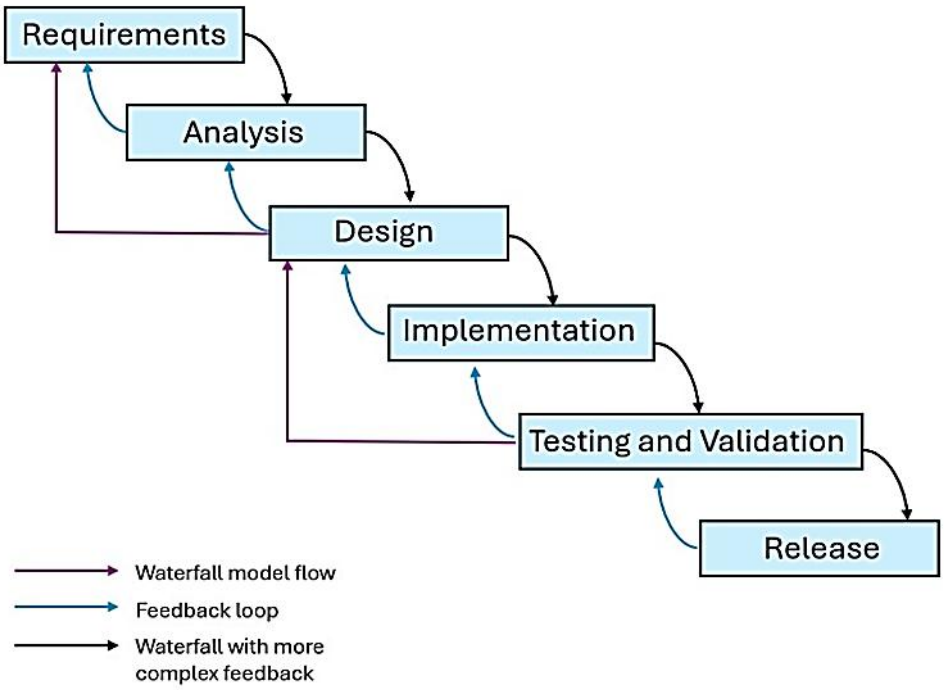


Figure 1.1: Waterfall Development Methodology

The Waterfall model consists of the following phases:

- **Requirement Analysis -** Detailed requirements were gathered by analyzing the needs of customers, administrators, and delivery staff. Both functional and non-functional requirements were clearly defined.
- **System Design -** Based on the requirements, the overall system architecture, database structure, and process flow were designed using structured techniques such as Data Flow Diagrams, ER diagrams, and flowcharts.

- **Implementation -** The system was developed using React for the frontend, Node.js/Express for backend processing, MySQL for database management, and the ml-random-forest JavaScript library for the machine learning module. Each module was implemented according to the predefined design.
- **Testing -** The completed system was tested through module-level functional testing and system testing to verify that the implemented functionalities, including the ML prediction pipeline, worked according to the specified requirements.
- **Maintenance -** Maintenance is considered as a future phase involving error correction, performance improvement, ML model retraining as more delivery data becomes available, and future system updates.

The Waterfall model ensures better documentation, clear phase separation, and controlled development, making it appropriate for this project.

1.6 Report Organization

This report is organized to describe the analysis, design, development, and implementation of QuickShip using a structured approach and the Waterfall model.

- **Chapter 1: Introduction —** provides an overview of the project, including background information, problem statement, objectives, scope and limitations, development methodology, and report structure.
- **Chapter 2: Background Study and Literature Review —** examines relevant theories, technologies, and existing courier management systems, and reviews related research in the courier and logistics domain.
- **Chapter 3: System Analysis and Design —** details system requirements, feasibility study, and structured system design using DFDs, ER diagrams, architecture, database schema, and the core algorithms.
- **Chapter 4: Implementation and Testing —** describes the tools and technologies used, module implementation details, and testing strategies applied to validate the system.
- **Chapter 5: Conclusion and Future Recommendations —** summarizes the project outcomes and proposes possible future improvements to the system.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Courier management systems are computerized solutions designed to manage parcel booking, delivery tracking, staff coordination, and payment handling in courier and logistics services. With the rapid growth of e-commerce and online transactions, the demand for fast, reliable, and transparent courier services has increased significantly. Manual courier systems are no longer sufficient to handle the growing volume of parcels and customer expectations.

A web-based courier management system helps automate daily operations such as customer registration, parcel entry, delivery assignment, and status updates. By using a centralized relational database, courier companies can store and manage parcel information efficiently, reducing errors caused by manual record keeping. Customers benefit from status tracking and digital payment options, while administrators gain better control over courier operations through analytics and an audit trail.

This project adopts a structured system development approach, which focuses on process flow, data movement, and functional decomposition. Structured analysis techniques such as requirement analysis, Data Flow Diagrams (DFDs), and normalized database design are used to understand how data moves through the system and how processes interact.

The Waterfall development model is applied in this project due to its systematic and sequential nature. Each phase — requirement analysis, design, implementation, testing, and maintenance — is completed step by step, ensuring clear documentation and better control over development. This approach is suitable for academic projects where system requirements are well-defined at the beginning.

Beyond the standard three-tier web stack, QuickShip incorporates a small but genuine applied machine-learning component. Ensemble tree methods such as Random Forest are well suited to small, tabular, real-world datasets because they are robust to noise, require little feature engineering, and remain interpretable enough for an academic viva. Rather than reporting a single train/test split — which is unstable on a small dataset —

the project applies k-fold cross-validation, a standard technique in applied machine learning for producing a more defensible estimate of model generalization when data is limited.

Modern web technologies play a crucial role in developing such systems. React is used to build an interactive and responsive frontend, Node.js with Express handles backend logic, REST APIs, and business rules, and MySQL manages structured relational data such as users, parcels, payments, and delivery records. Integration of eSewa enables secure and convenient online payments, improving user experience and operational efficiency.

2.2 Literature Review

In recent years, technological advancements have significantly transformed the logistics and courier industry by introducing automated and web-based management systems. Several studies have explored the use of information systems and, increasingly, applied machine learning to improve courier operations, delivery tracking, and customer satisfaction.

Aravind et al. proposed an Automated Courier Management System, demonstrating how automation of parcel booking, delivery assignment, and tracking reduces manual errors and improves operational efficiency. This research provides a foundation for systems such as QuickShip, which rely on database-driven automation to manage courier information effectively.

Deshmukh et al. emphasized the role of status tracking and cloud-based databases in enhancing operational transparency and customer satisfaction. Their study shows that integrating cloud technologies allows customers and administrators to access accurate parcel status updates instantly, thereby improving overall service quality.

Rabuan et al. presented a Courier Tracking Management System with Notifications using Barcode Scanners, which significantly improved delivery reliability and operational efficiency by automating status updates and reducing manual workload. QuickShip applies the same principle using QR codes rather than barcodes, generated at booking time and scannable by the customer for instant tracking.

Durai et al. proposed an Automatic Courier Management System integrating parcel registration, tracking, and delivery monitoring to handle large parcel volumes

efficiently while minimizing operational costs and errors. The automation principles outlined in this study are directly applicable to QuickShip's parcel-assignment and booking-history modules.

Patel et al. introduced the Go-Courier Centralized Courier Management System, a web-based platform that streamlines operations using status tracking and a centralized database, demonstrating improved service delivery and reduced operational errors. This research supports QuickShip's approach to centralized processing of courier operations.

Betrand et al. developed a Logistics Web Application for Parcel Tracking, showing that web-based platforms facilitate better management of parcel flow and enhance user accessibility to delivery status information. This supports the inclusion of an online, timeline-based tracking view in QuickShip.

Hiuredhy et al. explored an efficient tracking and delivery system for logistics, focusing on optimization of parcel delivery processes to improve operational performance and reduce delays. Their study highlights the importance of workflow optimization, which QuickShip addresses through its Weighted Staff Recommendation algorithm.

Asika's research on a web-based courier management system emphasizes the benefits of web-enabled tracking and administrative controls, which reduce manual data entry, enhance operational flexibility, and improve system responsiveness. These findings directly support QuickShip's web-based, role-based architecture.

Ammulu and Madhu Sudhan Reddy's Online Courier Management System study demonstrates how integrated courier platforms can facilitate parcel booking, hub management, and data processing through web technologies, thereby improving system scalability and accessibility.

Finally, Jaiswal et al. explored AI-based courier tracking and automation systems, showing that intelligent algorithms can automate tracking decisions, improve accuracy, and enhance system responsiveness. QuickShip builds directly on this direction with its Random Forest delivery-time prediction module, while being careful to evaluate and report the model's real accuracy rather than assuming intelligent methods are automatically reliable.

Collectively, these studies highlight the growing importance of automated, web-based, and intelligent courier management systems. They validate the design principles behind

platforms like QuickShip, particularly in parcel automation, status tracking, centralized operations, staff coordination, enhanced customer access, and — increasingly — the responsible application of machine learning to operational prediction problems.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis was carried out to understand the requirements, users, processes, and overall structure of QuickShip before and during implementation. QuickShip is a smart courier management system developed to simplify parcel booking, delivery management, parcel tracking, payment processing, and delivery-time estimation.

The system provides separate functions for customers, delivery staff, and administrators. Customers can book parcels, calculate delivery charges, track shipments, make payments, and provide feedback. Delivery staff can view assigned parcels, update delivery statuses, and confirm successful deliveries or record unsuccessful delivery attempts. Administrators can manage users, parcels, staff assignments, reports, and other system activities.

QuickShip also incorporates a Weighted Staff Recommendation algorithm to assist in staff assignment and a Random Forest regression model for delivery-time estimation. The system analysis identified the functional requirements and established the foundation for system modelling, design, and implementation.

3.1.1 Requirement Analysis

Requirement analysis helps in identifying what the system should do and how it should perform. The requirements of QuickShip are categorized into functional and non-functional requirements.

3.1.1.1 Functional Requirements

- **User Registration and Authentication** - The system shall allow users to register and log in using JWT-based authentication with bcrypt password hashing. Three roles are supported: client, staff, and admin. Staff accounts are managed by administrators. The password-reset mechanism is simplified and does not implement email ownership verification through a reset link or OTP.
- **Courier Booking** - Clients shall be able to book courier parcels by providing receiver details, parcel weight, parcel description, delivery type, and priority. The available delivery types are Air and Vehicle, while the priority options are Standard and Express. The system calculates the delivery cost and provides a

delivery-time estimate generated either by the trained Random Forest model or the rule-based fallback.

- **Parcel Tracking** - Clients shall be able to track parcels using a tracking ID or by scanning the parcel QR code using a device camera. The system displays the current parcel status together with a chronological tracking timeline, including relevant delivery-attempt and rescheduling events.
- **Staff Assignment -** Administrators shall be able to assign parcels to delivery staff manually or use the system's Weighted Staff Recommendation mechanism. The recommendation is based on workload, distance, availability, and historical delivery success rate. The recommendation score and assignment reason are available for administrative review.
- **Delivery Status Update -** Delivery staff shall be able to update the parcel status through supported stages such as Picked Up, In Transit, and Out for Delivery. Staff can confirm successful delivery by entering receiver information and remarks. They can also record failed delivery attempts using predefined reasons and optionally provide a reschedule date.
- **Payment -** Clients shall be able to select eSewa for online payment or Cash on Delivery. The system stores the payment method, amount, transaction reference where applicable, and payment status.
- **Feedback -** Clients shall be able to submit star ratings and written reviews after a parcel has been delivered.
- **User Management -** Administrators shall be able to manage user accounts and staff users, including creating staff accounts and updating or removing user records as permitted by the system.
- **Machine Learning Delivery Prediction** - Administrators shall be able to train or retrain the Random Forest delivery-time prediction model when sufficient historical delivery data is available. The system displays evaluation metrics generated from cross-validation. The Delivery Accuracy Report compares stored predicted delivery time with actual delivery time for delivered parcels.

- **Activity Logging** - The system records selected staff and administrator parcel-related activities such as parcel assignment, status updates, delivery confirmation, and failed-delivery-attempt events.
- **Virtual Help Assistant** - The system shall provide a rule-based Virtual Help Assistant that responds to predefined common courier-related queries such as parcel booking, tracking, payment, delivery status, delivery charges, and other supported user intents.



Figure 3.1: Use Case Diagram of QuickShip

3.1.1.2 Non-Functional Requirements

- **Security** - The system uses JWT-based authentication, bcrypt password hashing, and role-based authorization middleware to protect restricted operations. The simplified password-reset mechanism does not verify ownership of the submitted email address.
- **Performance -** The system should respond within a reasonable time to common operations such as parcel booking, tracking, cost calculation, and dashboard requests.
- **Scalability -** The normalized relational database and stateless REST API provide a basis for future expansion in the number of users, parcels, and delivery staff.
- **Usability -** The interface should be responsive, clear, and easy to use for clients, staff, and administrators across supported devices.
- **Reliability -** The system should maintain consistent parcel and delivery information and provide a clearly labeled rule-based fallback ETA when the Random Forest model cannot be used because sufficient data is unavailable.

3.1.2 Feasibility Analysis

The feasibility study of QuickShip was conducted to evaluate whether the proposed system is practical and achievable within the available resources, time, and technical constraints. The feasibility analysis covers technical, operational, economic, and schedule aspects.

- **Technical Feasibility -** QuickShip uses React.js and Tailwind CSS for the frontend, Node.js and Express.js for backend processing, MySQL through Sequelize for database management, and the ml-random-forest JavaScript library for the machine-learning module. The Random Forest model runs within the Node.js backend and does not require a separate Python service or external machine-learning API. The technologies used are suitable for implementing a full-stack academic project and can operate on standard development hardware.

- **Operational Feasibility -** The system is designed to simplify courier operations through role-based access. Clients can perform booking, payment, tracking, and feedback activities. Staff can manage assigned parcels and delivery status, while administrators can manage users, assignments, reports, and system operations. Because each role is provided with an appropriate interface and set of permissions, the system is operationally feasible.

- **Economic Feasibility** -The project uses open-source development technologies and libraries, minimizing software licensing costs. Development can be carried out using ordinary computer hardware and a standard web-development environment. By reducing manual record management and automating several courier activities, the system has the potential to reduce operational effort.

- **Schedule Feasibility -** The development schedule was planned around the academic project timeline. The Waterfall methodology provided a sequential framework for completing requirement analysis, design, implementation, testing, and documentation within the available project period.

  GANTT CHART

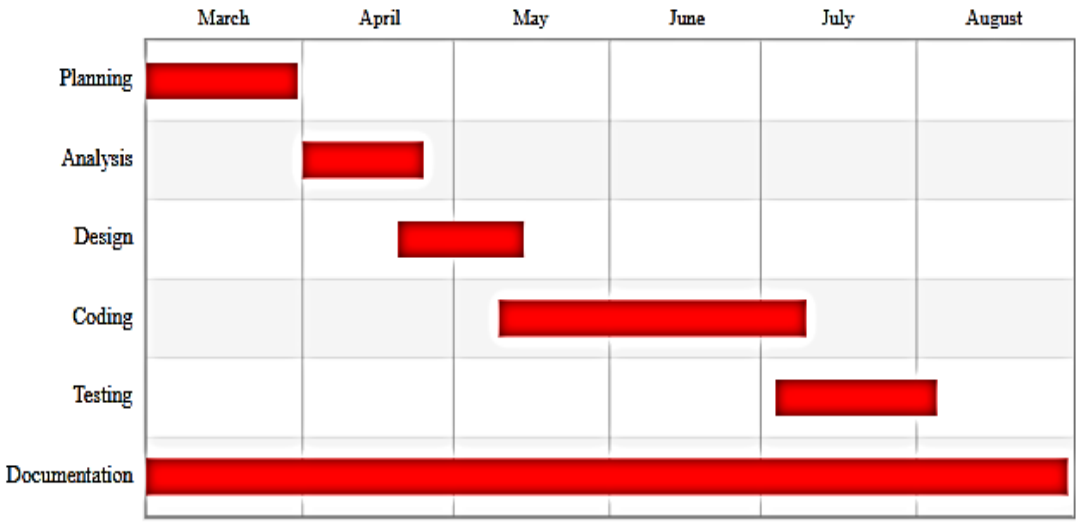


Figure 1.2: GANTT Chart of QuickShip Project Development

3.1.3 System Object Modelling Using Class and Object Diagrams

System object modelling represents the main entities, their attributes, and their relationships. It helps explain how information is organized within the QuickShip system and provides a foundation for database design and backend implementation.

The principal entities include User, Location, Parcel, Delivery, DeliveryAttempt, Payment, TrackingHistory, Feedback, and ActivityLog.

The User entity represents customers, delivery staff, and administrators. The Location entity stores source and destination location information. The Parcel entity represents booking details, including parcel weight, delivery type, priority, cost, status, and tracking information.

The Delivery entity represents delivery assignment and related completion information. DeliveryAttempt records unsuccessful delivery attempts. Payment stores payment-related information, while TrackingHistory maintains parcel-status events. Feedback stores customer ratings and comments, and ActivityLog records relevant system activities.

The relationships among these entities support the major courier operations. A customer can create parcel bookings, and a parcel can be associated with source and destination locations, delivery information, payment information, and tracking history. Additional records may be associated with delivery attempts, feedback, and activity logs according to the implemented relationships.

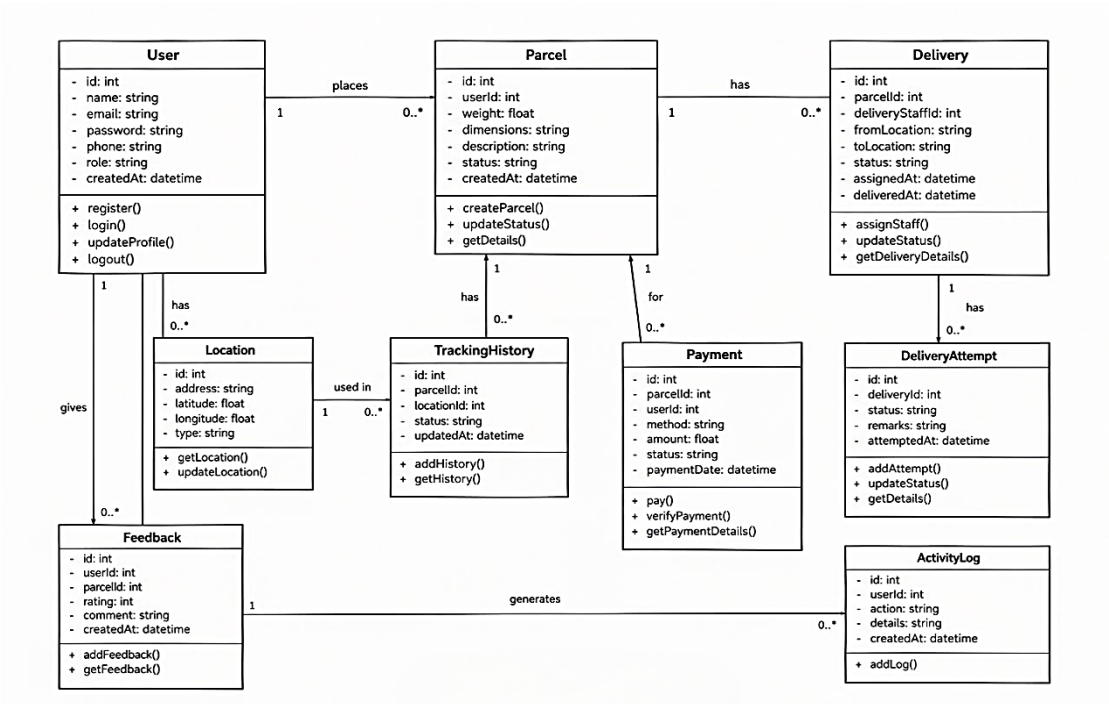


Figure 3.3: Class Diagram of QuickShip

An object diagram represents example instances of classes at a particular point in time. For example, a customer object may be associated with a parcel object, two location objects, a delivery object, and a payment object. A tracking-history object may represent a status update for the same parcel.

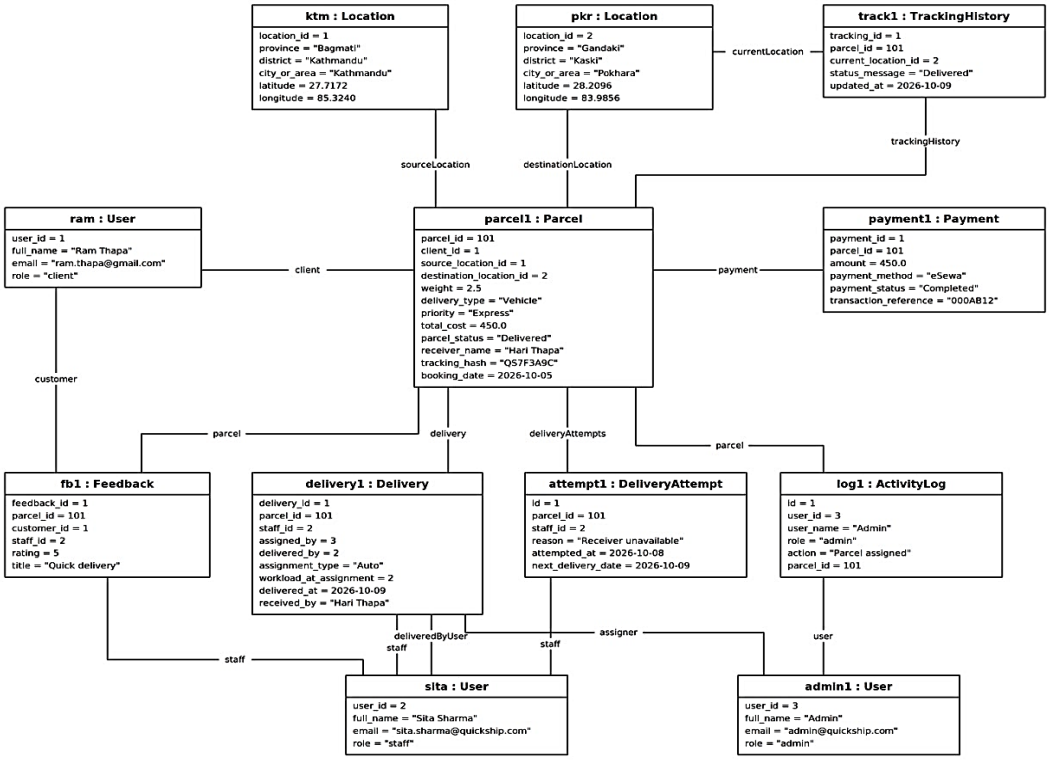


Figure 3.4: Object Diagram of QuickShip

The class and object diagrams provide structural views of the application and help explain how its main entities are connected during parcel booking and delivery management.

3.1.4 Dynamic Modelling Using State and Sequence Diagrams

Dynamic modelling describes how the system behaves over time and how its components interact during important operations.

The state diagram represents the parcel lifecycle and the permitted transitions between parcel statuses. Depending on the implemented workflow, a parcel may progress through states such as Booked, Assigned, Picked Up, In Transit, Out for Delivery, and Delivered. Failed delivery attempts, rescheduling, and cancellation should be represented according to the status transitions supported by the actual application.

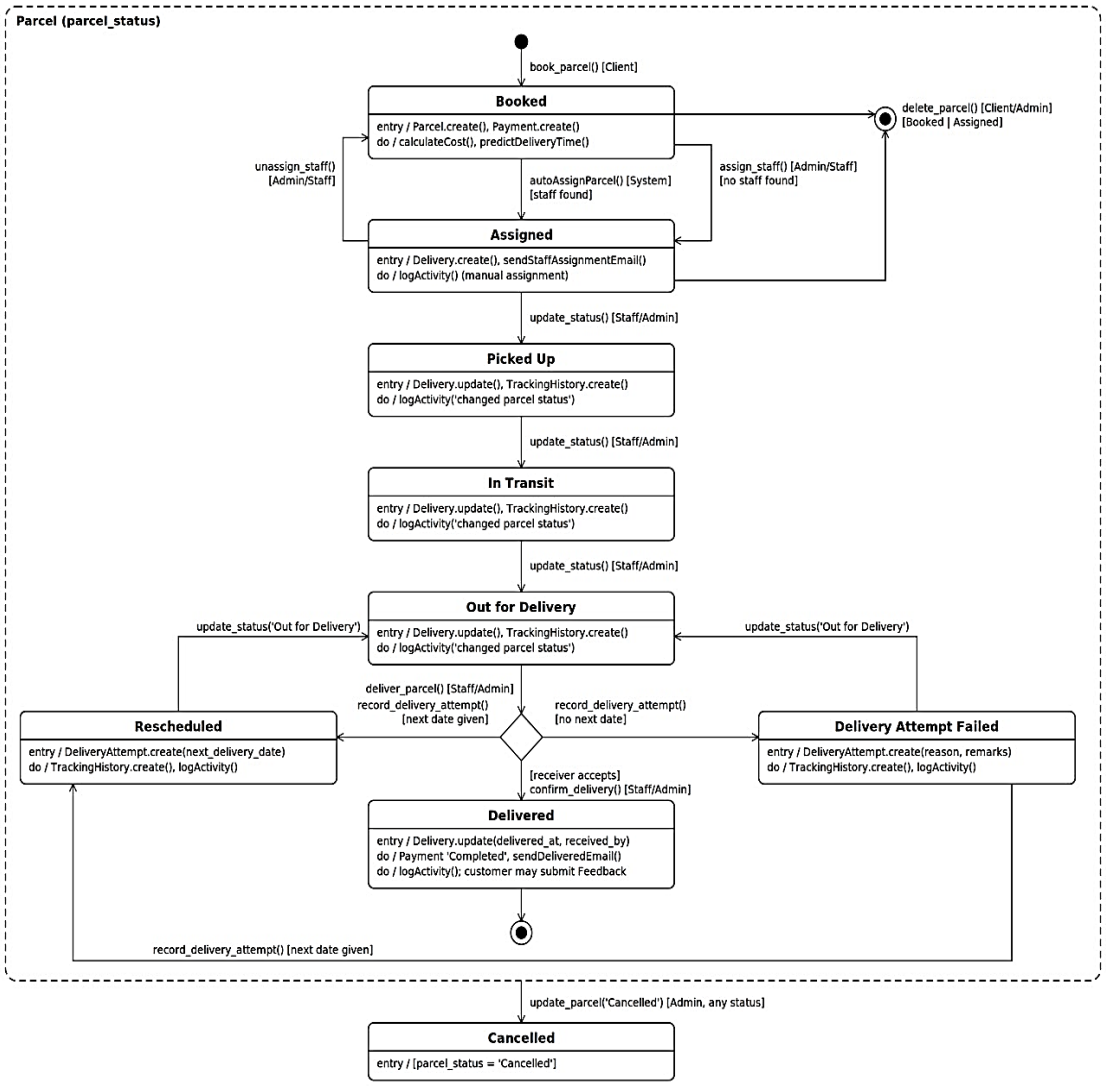


Figure 3.5: State Diagram of QuickShip Parcel Delivery Lifecycle

The sequence diagram represents the order of communication between users and system components. During parcel booking, for example, the customer submits parcel details through the frontend. The backend validates the request, performs the required calculations, stores the relevant information in the database, and returns the result to the frontend.

Sequence diagrams may also represent authentication, staff assignment, payment processing, delivery-status updates, and parcel tracking, provided that the diagrams correspond to the implemented workflows.

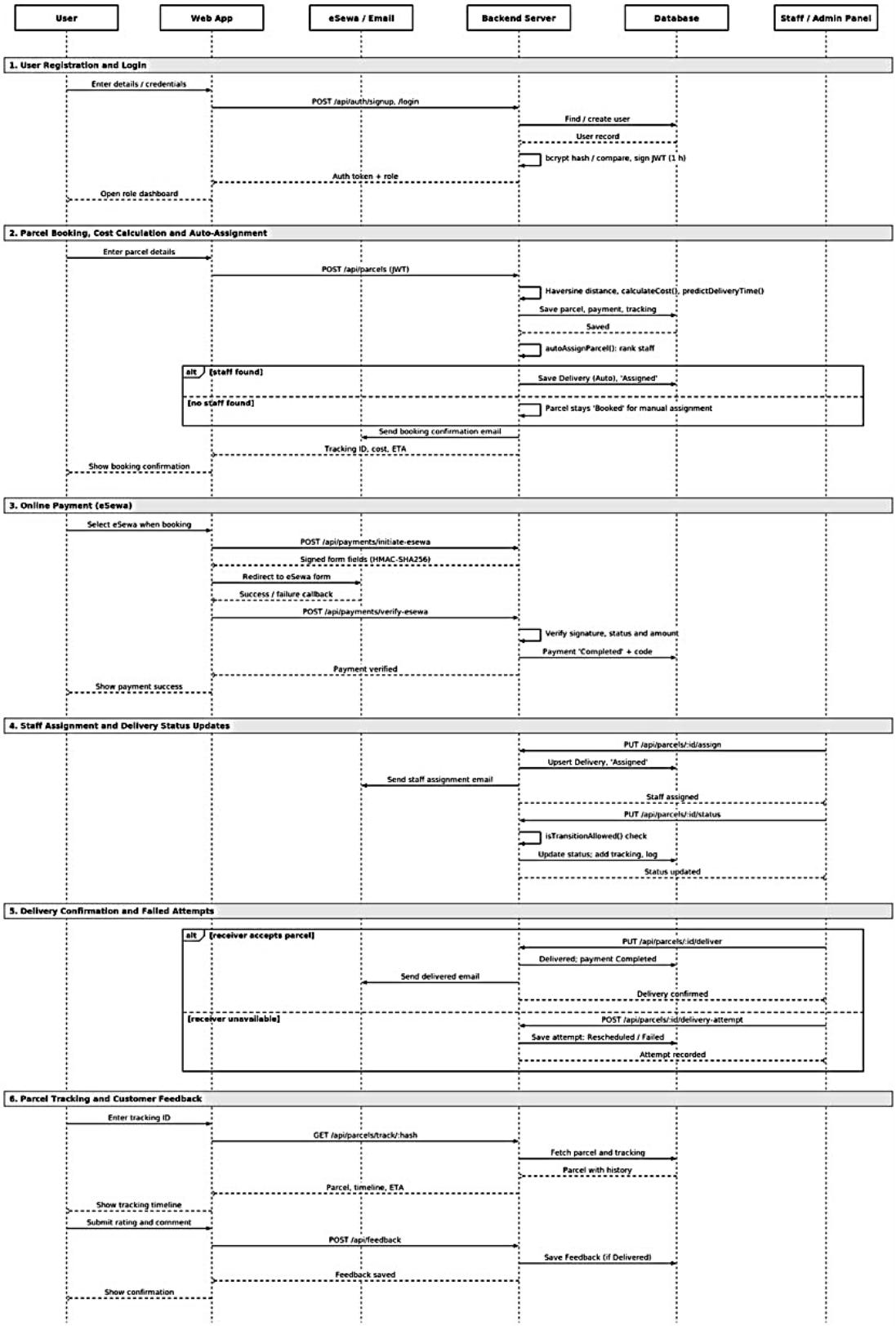


Figure 3.6: Sequence Diagram of the QuickShip

These diagrams help explain how system objects change state and how the frontend, backend, database, and supporting services coordinate their operations.

3.1.5 Process Modelling Using Activity Diagrams

Process modelling represents the sequence of activities and decisions involved in completing a system operation. Activity diagrams provide a visual representation of the workflows followed by customers, staff, and administrators.

The parcel-booking workflow begins when a customer enters parcel details and selects the source and destination locations. The system validates the information, calculates the delivery charge, obtains a delivery-time estimate, and processes the booking request. The system then returns the booking result.

Other activity diagrams may represent staff assignment, parcel tracking, payment processing, delivery confirmation, and delivery rescheduling. These diagrams should reflect the actual sequence of actions implemented in QuickShip.

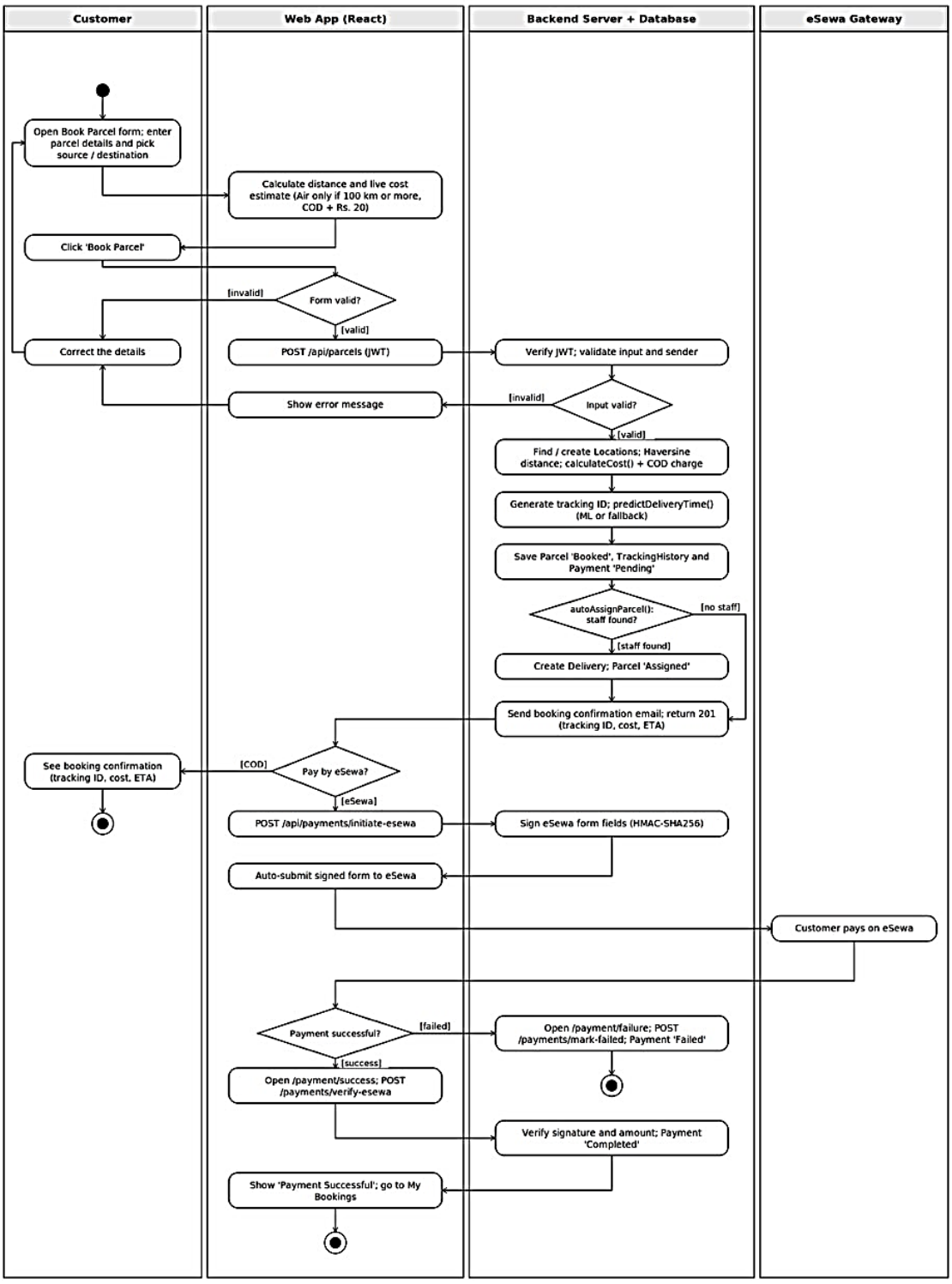


Figure 3.7: Activity Diagram of QuickShip Parcel Booking

Activity modelling helps clarify the steps, decisions, and system responses involved in the main courier operations.

3.2 System Design

The system design of QuickShip defines the overall architecture, major components, technologies, and interactions of the application.

QuickShip follows a client-server architecture. The frontend provides the user interface, the backend processes business logic and API requests, and the database stores and manages application information.

The frontend was developed using React.js and Vite, with Tailwind CSS for styling. React Router supports navigation, while Axios enables communication with the backend. The backend uses Node.js and Express.js to handle authentication, parcel booking, delivery management, payment-related operations, tracking, and other system functions. Sequelize ORM manages interaction with the MySQL database.

Supporting technologies include JWT and bcrypt for authentication, the Haversine formula for geographical distance calculation, the Weighted Staff Recommendation algorithm for staff assignment support, and the Random Forest regression service for delivery-time estimation. QR code functionality, eSewa sandbox integration, and Nodemailer support additional features.

3.2.1 Mockups and High-Level Designing

Mockups and high-level design describe the organization and appearance of the user interfaces. The QuickShip interface was organized according to the responsibilities of customers, delivery staff, and administrators.

The customer interface provides access to parcel booking, delivery-cost calculation, booking history, tracking, payment-related information, profile management, feedback, and the Virtual Help Assistant.

The staff interface provides access to assigned parcels, parcel details, delivery-status controls, delivery history, feedback-related information, and the staff Virtual Help Assistant.

The administrative interface provides access to customer and staff management, parcel management, staff assignment, analytics, prediction-related information, feedback, activity logs, and the administrative Virtual Help Assistant.

The following figures should contain actual screenshots from the implemented system or original interface mockups.

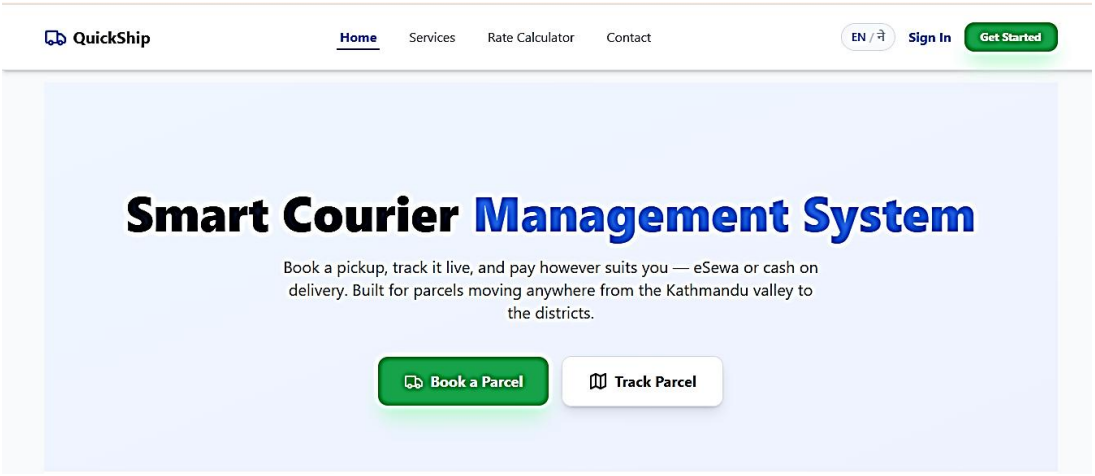


Figure 3.8: Landing Page of QuickShip

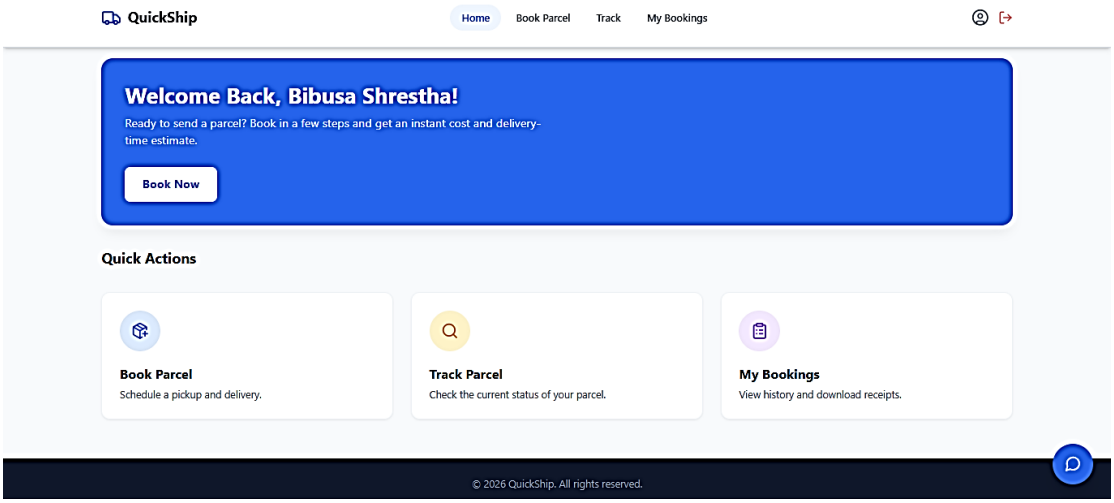


Figure 3.9: Client Dashboard of QuickShip

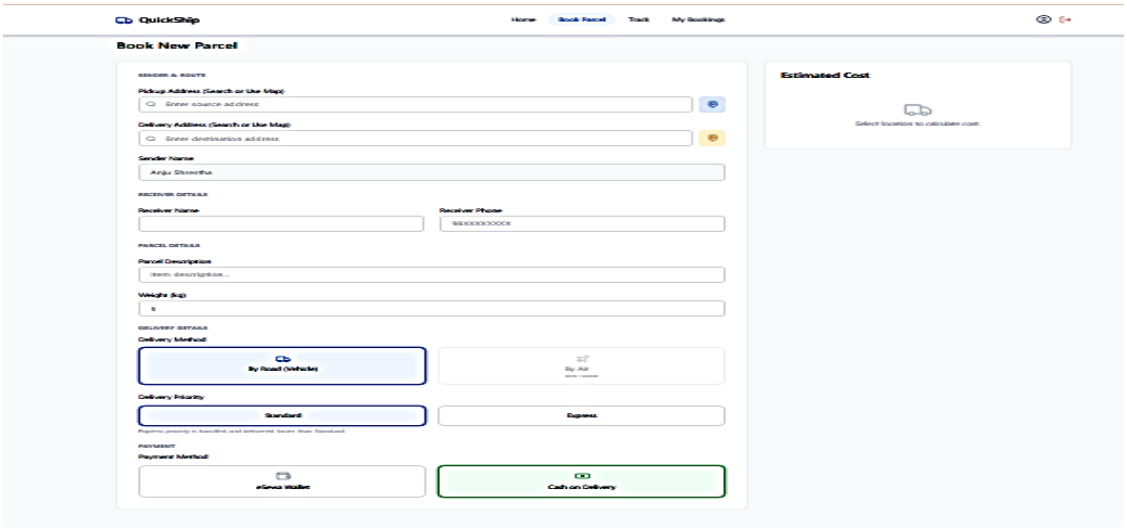


Figure 3.10: Parcel Booking Page of QuickShip

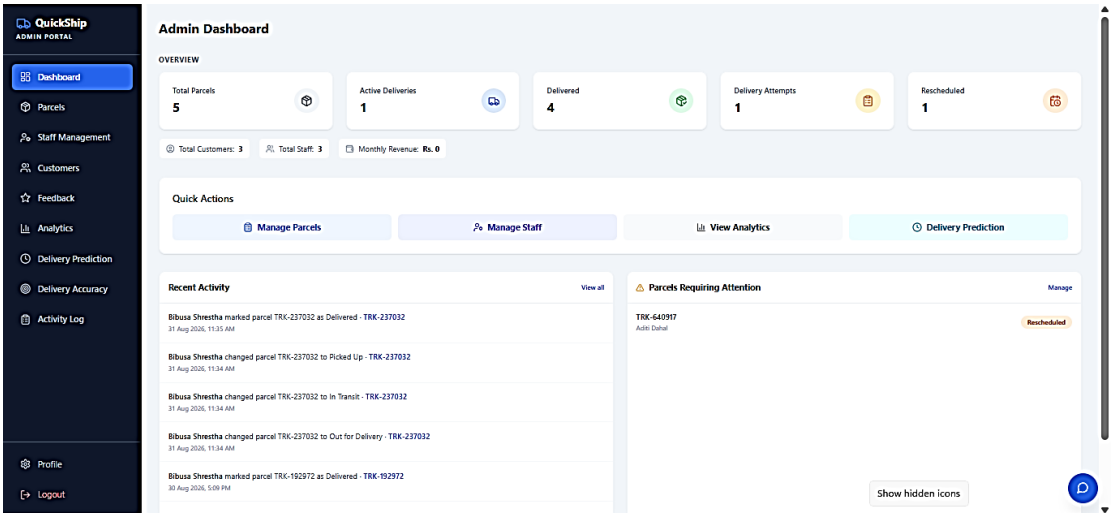


Figure 3.11: Admin Dashboard of QuickShip

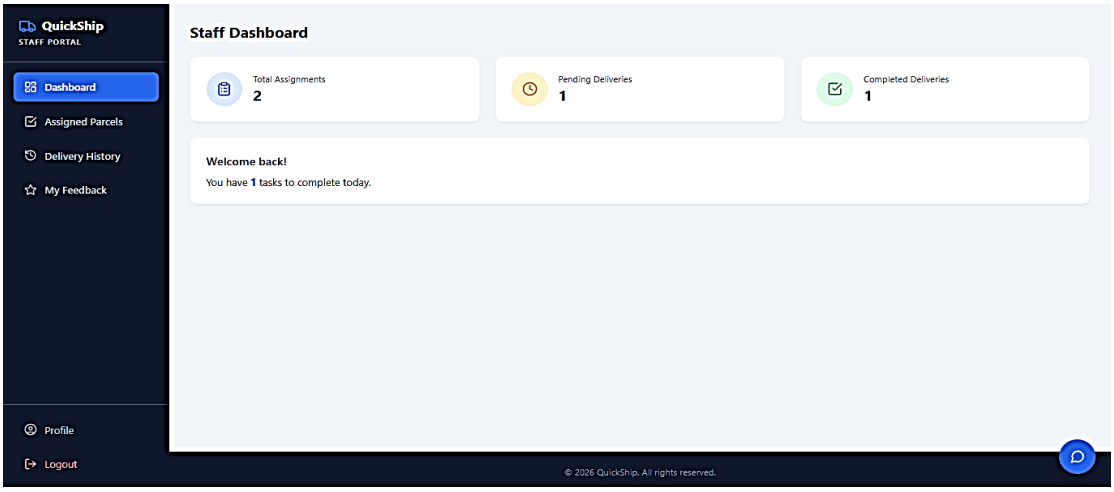


Figure 3.12: Staff Dashboard of Quickship

3.2.2 Refinement of Class, Object, State, Sequence, and Activity Diagrams

The class, object, state, sequence, and activity diagrams should be refined to maintain consistency with the implemented system.

The class diagram represents the main entities and their relationships. The object diagram illustrates example instances of these entities. The state diagram represents parcel-status transitions, while the sequence diagram explains communication between system components. The activity diagram represents the workflow of parcel booking and other supported operations.

The final diagrams should use consistent entity names, status values, and relationships. They should correspond to the actual database models, API operations, and application workflows. This refinement helps make the design easier to understand and supports consistency between the documentation and implementation.

3.2.3 Component Diagrams

A component diagram represents the major software components and their dependencies. QuickShip is organized into frontend, backend, database, and supporting service components.

The React frontend contains interfaces for customers, staff, and administrators. It communicates with the Express backend through HTTP API requests. The backend validates requests, applies authentication and authorization, executes business logic, and returns responses to the frontend.

The MySQL database stores application information, while Sequelize ORM manages database operations. The authentication component supports JWT verification and password hashing. The parcel-management component handles booking and tracking, while the delivery-assignment component implements the Weighted Staff Recommendation algorithm.

The delivery-prediction component processes historical delivery records and provides estimates when a trained Random Forest model is available. Other components support distance and cost calculation, payment processing, QR code functionality, email notifications, feedback, analytics, and activity logging.

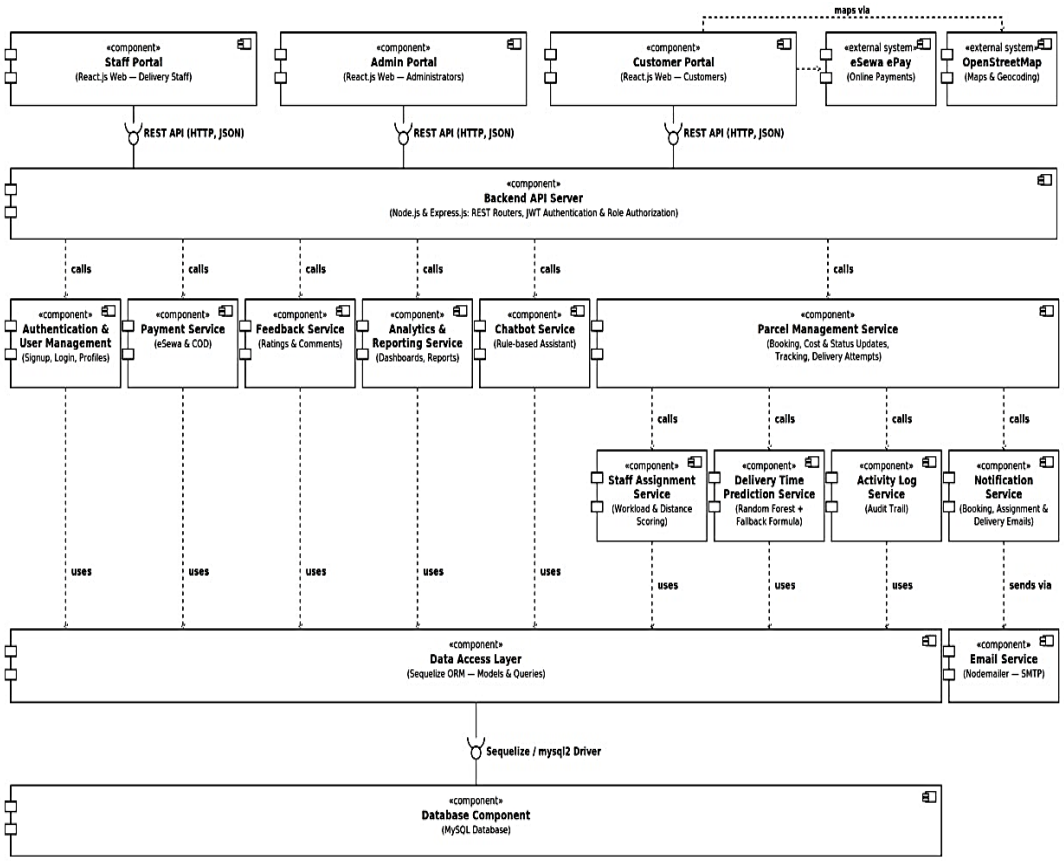


Figure 3.13: Backend Component Diagram of QuickShip

The component diagram should represent the actual relationships between the frontend, backend routes and services, database, and configured external integrations.

3.2.4 Deployment Diagrams

A deployment diagram represents the runtime environment and the communication paths between major system components.

QuickShip can operate with a web browser running the frontend, a Node.js runtime hosting the Express backend, and a MySQL database storing application information. The frontend communicates with the backend through API requests, and the backend accesses the database through Sequelize ORM.

The delivery-time prediction service operates within the backend. External services may include the eSewa sandbox and a configured SMTP service for email notifications. QR code functionality is provided through application libraries.

During development, the frontend, backend, and database can operate in a local development environment. Production deployment would require suitable hosting, connectivity, configuration, and security arrangements.

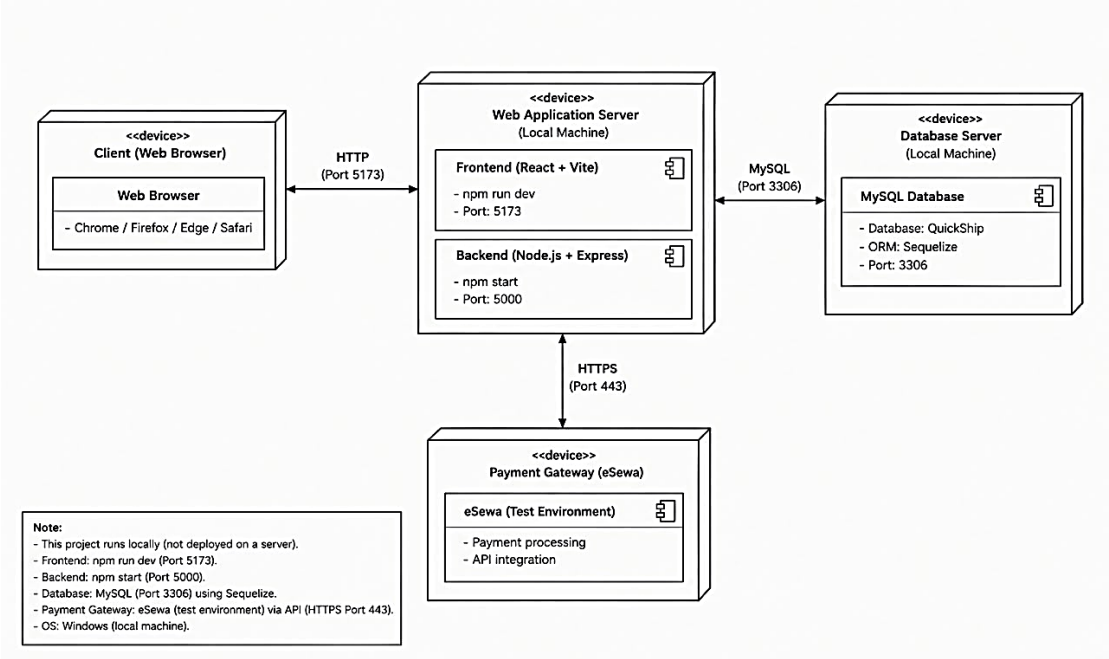


Figure 3.14: Deployment Diagram of QuickShip

The deployment diagram provides an overview of the runtime environment and the communication between users, application components, the database, and external services.

3.3 Algorithm Details

QuickShip uses algorithms to support distance calculation, delivery-cost calculation, staff recommendation, and delivery-time estimation. These algorithms help automate selected operations and provide consistent processing of parcel-related information.

3.3.1 Distance Calculation — Haversine Algorithm

The Haversine algorithm is used to calculate the geographical distance between two points using latitude and longitude values.

In QuickShip, the distance is used in delivery-cost calculation and as an input feature for delivery-time prediction.

Steps

Step 1: Retrieve latitude and longitude values for the source and destination locations.

Step 2: Convert the coordinates from degrees to radians.

Step 3: Calculate the differences between the latitude and longitude values.

Step 4: Apply the Haversine formula.

Step 5: Calculate the central angular distance.

Step 6: Multiply the result by the Earth's radius, approximately 6,371 km, to obtain the distance in kilometers.

3.3.2 Cost Calculation Algorithm

The cost-calculation algorithm determines the delivery cost according to delivery method, parcel weight, distance, and the applicable payment option.

Steps

Step 1: Identify the selected delivery method: Vehicle or Air.

Step 2: Retrieve the applicable base charge, weight rate, and distance rate.

Step 3: Calculate the weight charge.

Step 4: Calculate the distance charge using the Haversine distance.

Step 5: Add the base charge, weight charge, distance charge, and applicable COD charge.

Step 6: Apply the rounding rule used by the system and store the final cost.

The resulting cost is displayed to the client during booking and on the invoice.

3.3.3 Weighted Staff Recommendation Algorithm

Rather than assigning parcels to staff purely by availability or in round-robin order, QuickShip uses a deterministic, rule-based Weighted Staff Recommendation algorithm. It does not use machine learning for staff selection.

Eligible staff members are evaluated using four factors:

Table 3.1: Weighted Staff Recommendation — Scoring Factors

Factor	Weight	How it is measured
Workload	35%	Number of currently active parcels assigned to the staff member (fewer active parcels scores higher).
Distance	25%	Haversine distance from the staff member's most recent active delivery destination to the new parcel's pickup point.
Availability	15%	Whether the staff member's active workload is below the busy threshold (4 active parcels).
Success Rate	25%	Ratio of delivered to total assigned parcels; a staff member with no history yet is given a neutral 0.85 rather than being unfairly penalized or boosted.

The weighted factors are combined into a final score. The highest-scoring eligible staff member is recommended to the administrator.

The administrator can accept the recommendation or manually assign another staff member. The system records whether the final assignment was manual or automatic and stores the assignment reason.

Steps

Step 1: Retrieve eligible staff users.

Step 2: Calculate workload score.

Step 3: Calculate distance score.

Step 4: Calculate availability score.

Step 5: Calculate historical success-rate score.

Step 6: Apply the predefined weights.

Step 7: Calculate the final weighted score.

Step 8: Rank the eligible staff.

Step 9: Recommend the highest-scoring candidate.

Step 10: Allow the administrator to accept or override the recommendation.

3.3.4 Random Forest Delivery-Time Prediction

QuickShip contains a Random Forest Regressor implemented using the ml-random-forest JavaScript package. The model runs within the Node.js backend and predicts delivery time in hours using historical delivered-parcel records.

Table 3.1: Features and Target

Feature	Source
Distance (km)	Haversine formula between source and destination coordinates
Parcel weight (kg)	Entered by the client at booking time
Delivery type	Air, Vehicle
Delivery priority	Express, Standard

The prediction target is actual delivery time in hours calculated from booking time to recorded delivery completion time.

Training Process

Each eligible delivered parcel becomes a training record. The system requires at least 15 usable delivered parcels before training is performed. If fewer records are available, model training is refused and the reason is reported to the administrator.

A deterministic shuffle is used for reproducibility. After cross-validation, the production model is trained using the available historical records.

Cross-Validation

The system uses k-fold cross-validation for model evaluation. Each record receives an out-of-fold prediction from a model that did not use that record during training.

The evaluation metrics include:

- Mean Absolute Error (MAE)

- Root Mean Squared Error (RMSE)
- R^2 where applicable

The Random Forest configuration includes parameters such as maxDepth: 8, minNumSamples: 4, and nEstimators: 100.

Fallback Prediction

When the model has not been trained or sufficient data is unavailable, QuickShip uses a clearly labeled rule-based fallback estimate based on distance and delivery type, with an Express-priority adjustment.

Each stored prediction identifies its source as either **ml** or **fallback**.

The administrator can retrain the model as additional historical delivery data becomes available.

QR-Based Tracking and Delivery Status

QuickShip also supports parcel tracking through QR codes.

Steps

Step 1: A tracking identifier is associated with the parcel during booking.

Step 2: The system generates a QR code for the parcel.

Step 3: The client opens the tracking interface.

Step 4: The client enters the tracking identifier or scans the QR code using the device camera.

Step 5: The tracking request is sent to the backend.

Step 6: The backend retrieves the corresponding parcel and tracking-history records.

Step 7: The current parcel status and chronological tracking timeline are displayed.

Step 8: Relevant delivery-attempt and rescheduling events are displayed where applicable.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

The implementation phase of QuickShip involved converting the system design into a functional web-based application. The implemented modules include authentication, parcel booking, cost calculation, staff recommendation and assignment, parcel status tracking, delivery confirmation, failed delivery attempts and rescheduling, payment, feedback, analytics, activity logging, email notifications, QR tracking, the Virtual Help Assistant, and the Random Forest delivery-time prediction pipeline.

The system was developed using modern web technologies, with React (Vite) for the frontend, Node.js with Express for backend processing, MySQL (via Sequelize) for database management, and the ml-random-forest package for the machine-learning module. The development followed the Waterfall model, where each stage was completed sequentially, ensuring clarity, proper documentation, and systematic progress.

4.1.1 Tools Used

- **Visual Studio Code (VS Code) —** used as the primary development environment for both frontend and backend development, with integrated terminal, debugging, and linting support.
- **draw.io —** used to design system diagrams such as Data Flow Diagrams, flowcharts, and the Entity Relationship diagram, helping visualize system processes and database structure during the design phase.
- **React.js (Vite) —** used to develop the frontend of the system, enabling interactive, componentized dashboards for client, staff, and admin roles with fast hot-module reloading during development.
- **Node.js —** used as the backend runtime environment to handle server-side logic, process client requests, manage REST APIs, run the Random Forest ML module, and communicate with the MySQL database.
- **Tailwind CSS —** used to style the frontend as a utility-first CSS framework, enabling rapid, consistent, and responsive UI development without writing extensive custom CSS.

- **Express.js —** used as the web application framework for Node.js to build the REST API, simplifying route handling, authentication middleware, and communication between the frontend and MySQL.
- **Sequelize ORM —** used to define models, associations, and migrations for the nine relational tables, and to keep the schema in sync automatically on server start.
- **MySQL —** used as the relational database management system for storing structured data such as user details, parcel information, delivery records, payments, tracking history, and feedback.
- **ml-random-forest —** a pure-JavaScript Random Forest implementation used to train and run the delivery-time prediction model directly inside the Node.js backend.
- **JWT & bcrypt —** used respectively for stateless authentication tokens and secure password hashing.
- **Nodemailer** — used to send booking confirmation, staff assignment, and delivery-completion emails through SMTP when email configuration is available.
- **Git and GitHub —** used for version control and to host the project repository, tracking development progress and preventing data loss.
- **Web Browsers (Brave and Google Chrome) —** used for testing and debugging the frontend, ensuring proper rendering, responsiveness, and functionality.

4.1.2 Implementation Details of Modules

The QuickShip Courier Management System was divided into different modules to simplify development and improve maintainability.

User Management Module

The user management module handles registration, login, and authentication of users. It supports three roles — admin, staff, and client. Role-based access control, enforced

through Express middleware that verifies the JWT and attaches the authenticated user to the request, ensures that each user can only access the features relevant to their role.

```
const verifyToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    if (!authHeader) return res.status(403).json({ message: 'No token provided' });

    const token = authHeader.split(' ')[1];
    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
        if (err) return res.status(401).json({ message: 'Unauthorized: Invalid token' });
        req.user = decoded;
        next();
    });
};

const authorize = (roles = []) => {
    return (req, res, next) => {
        if (!req.user) return res.status(401).json({ message: 'Unauthorized' });
        if (roles.length && !roles.includes(req.user.role)) {
            return res.status(403).json({ message: 'Forbidden: Insufficient permissions' });
        }
        next();
    };
};
```

Figure 4.1: User Management Module (Auth Middleware)

Parcel Booking Module

This module allows clients to book courier parcels by entering receiver details, parcel weight, description, delivery type, and priority. On submission, the system calculates the Haversine distance, cost breakdown, and a delivery-time estimate from either the trained Random Forest or the rule-based fallback. The booking is stored with a unique tracking hash, which is also used for QR-based tracking.

```
const haversine = (lat1, lon1, lat2, lon2) => {
    const R = 6371;
    const dLat = (lat2 - lat1) * Math.PI / 180;
    const dLon = (lon2 - lon1) * Math.PI / 180;

    const a =
        Math.sin(dLat / 2) ** 2 +
        Math.cos(lat1 * Math.PI / 180) *
        Math.cos(lat2 * Math.PI / 180) *
        Math.sin(dLon / 2) ** 2;

    return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
};

const calculateCost = (weight, distance, deliveryMethod) => {
    let baseRate = 50;
    let weightRate = 5;
    let distanceRate = 1;

    if (deliveryMethod === 'Air') {
        baseRate = 150;
        distanceRate = 2;
    }

    return Math.ceil(
        baseRate + weight * weightRate + distance * distanceRate
    );
};
```

Figure 4.2: Parcel Booking Module (Cost & ML ETA)

Weighted Staff Assignment Module

The staff assignment module allows the admin to assign or reassign parcels to delivery staff. The system can rank staff using workload, distance, availability, and success rate, while the admin can override the recommendation and assign manually. The assignment type and reasoning are stored for audit purposes.

```
const WEIGHTS = {
    workload: 0.35,
    distance: 0.25,
    availability: 0.15,
    successRate: 0.25
};

const workloadScore = 1 / (1 + workload);
const distanceScore = 1 / (1 + distanceKm / 10);
const availabilityScore = isAvailable ? 1 : 0.3;
const successScore = successRate;

const weightedScore =
    workloadScore * WEIGHTS.workload +
    distanceScore * WEIGHTS.distance +
    availabilityScore * WEIGHTS.availability +
    successScore * WEIGHTS.successRate;

return {
    staff_id: s.user_id,
    full_name: s.full_name,
    workload,
    distanceKm,
    successRate,
    isAvailable,
    score: Math.round(weightedScore * 100)
};
```

Figure 4.3: Weighted Staff Assignment Module

Delivery Status Update & Confirmation Module

This module allows staff to view assigned parcels and update delivery status through Picked Up, In Transit, and Out for Delivery. Successful delivery is confirmed by entering the receiver's name and optional remarks. When delivery fails, staff record a reason and may provide a next delivery date. The system then records either Delivery Attempt Failed or Rescheduled and adds the event to the tracking timeline.

```
const NEXT_STATUS = {
    'Booked': 'Assigned',
    'Assigned': 'Picked Up',
    'Picked Up': 'In Transit',
    'In Transit': 'Out for Delivery',
    'Delivery Attempt Failed': 'Out for Delivery',
    'Rescheduled': 'Out for Delivery'
};

const handleAdvanceStatus = async (id, newStatus) => {
    const token = localStorage.getItem('token');

    await axios.put(
        `http://localhost:5000/api/parcels/${id}/status`,
        { status: newStatus },
        {
            headers: {
                Authorization: `Bearer ${token}`
            }
        }
    );

    fetchTasks();
};
```

Figure 4.4: Delivery Status Update & Confirmation Module

Payment Module

The payment module facilitates secure online transactions using the eSewa payment gateway, as well as Cash on Delivery. It handles the integration between the application and the eSewa API, verifying the transaction signature and updating the database once payment succeeds, ensuring an accurate record of every financial transaction.

```
function buildSignature(fields, signedFieldNames, secret) {
    const message = signedFieldNames
        .split(',')
        .map(field => `${field}=${fields[field]}`)
        .join(',');

    return crypto
        .createHmac('sha256', secret)
        .update(message)
        .digest('base64');
}

const expectedSignature = buildSignature(
    decodedData,
    signed_field_names,
    ESEWA_SECRET
);

if (expectedSignature !== signature) {
    return res.status(400).json({
        error: 'Signature verification failed'
    });
}

await paymentRecord.update({
    payment_status: 'Completed',
    transaction_reference: transaction_code
});
```

Figure 4.5: Payment Module (eSewa Integration)

Parcel Tracking & QR Module

The tracking module allows clients to track their parcels using the tracking ID, typed in manually or scanned from the parcel's QR code via html5-qrcode. The system retrieves the latest status and the full tracking_history timeline from the database and displays it to the client, improving transparency and customer satisfaction.

```
const trackValue = async (value) => {
    const token = localStorage.getItem('token');

    const res = await axios.get(
        `http://localhost:5000/api/parcels/track/${encodeURIComponent(value.trim())}`,
        {
            headers: {
                Authorization: `Bearer ${token}`
            }
        }
    );

    setResults(
        Array.isArray(res.data) ? res.data : [res.data]
    );
};
```

```
const scanner = new Html5Qrcode('qr-scanner-region');

scanner.start(
    { facingMode: 'environment' },
    { fps: 10, qrbox: 220 },
    (decodedText) => {
        scanner.stop();
        onScan(decodedText);
    },
    () => {}
);
```

Figure 4.6: Parcel Tracking & QR Module

Random Forest ML Prediction Module

This module trains and serves the delivery-time prediction model. Training builds the dataset from delivered parcels, refuses to run below 15 records, trains the production model on the full dataset, and separately evaluates generalization with k-fold cross-validation before persisting both the model and its metrics to disk.

```
const MIN_TRAIN_SAMPLES = 15;

function encodeFeatures({
    distance,
    weight,
    deliveryType,
    priority
}) {
    return [
        Number(distance) || 0,
        Number(weight) || 0,
        deliveryType === 'Air' ? 1 : 0,
        priority === 'Express' ? 1 : 0
    ];
}

const forestOptions = {
    nEstimators: 100,
    seed: 42,
    maxFeatures: 0.8,
    treeOptions: {
        maxDepth: 8,
        minNumSamples: 4
    }
};

const foldModel = new RandomForestRegression({
    ...forestOptions,
    seed: 42 + fold
});

foldModel.train(
    trainFold.map(r => r.features),
    trainFold.map(r => r.actualHours)
);
```

Figure 4.7: Random Forest ML Prediction Pipeline

Predictions are stored on the parcel at booking time and surfaced on the Booking Confirmation panel, the Tracking page, the Booking Details modal, the Admin ML Prediction page (model status and metrics), and the Delivery Accuracy Report (predicted vs. actual vs. difference, per parcel).

4.2 Testing

Testing ensures that the system works correctly and meets all functional requirements. Both unit testing and system testing were used during the development of QuickShip.

4.2.1 Test Cases for Unit Testing

Unit testing was performed on individual modules to ensure that each module functions correctly. For example, login validation, parcel booking, cost calculation, and status-update functions were tested independently. Errors found during unit testing were fixed before moving to the next phase.

User Registration / Login

Table 4.1: Test Case for User Registration / Login

Test ID	Description	Input	Expected Output	Result
TC-UR-1	Open signup page	Signup	QuickShip Signup page loads	Pass
TC-UR-2	Register account	Full name, phone, email, password	Account created; redirect to login	Pass
TC-UR-3	Login with invalid credentials	Invalid email/password	Error: “Invalid credentials”	Pass
TC-UR-4	Login with valid credentials	Valid email/password	Redirect to role-based dashboard	Pass
TC-UR-5	Reset password with registered email	Registered email + new password	Account password updated; no email verification step	Pass

Book Parcel

Table 2.2: Test Case for Book Parcel

Test ID	Description	Input	Expected Output	Result
TC-BP-1	Open booking page	Customer/book	Parcel booking form loads	Pass
TC-BP-2	Submit with empty fields	Empty required inputs	Validation error shown	Pass

TC-BP-3	Submit with valid data	All required fields filled	Parcel booked; tracking ID + ML ETA shown	Pass
TC-BP-4	Pay with eSewa	Select eSewa, complete payment	Payment completed successfully	Pass

Parcel Status Tracking

Table 4.3: Test Case for Parcel Status Tracking

Test ID	Description	Input	Expected Output	Result
TC-PT-1	Open tracking page	Customer/track	Tracking page loads	Pass
TC-PT-2	Enter invalid tracking code	Invalid code	"Parcel Not Found" message	Pass
TC-PT-3	Enter valid tracking code	Valid code / QR scan	Status and timeline displayed	Pass

Booking History

Table 4.4: Test Case for Booking History

Test ID	Description	Input	Expected Output	Result
TC-BH-1	Open bookings page	Customer/bookings	Booking history page loads	Pass
TC-BH-2	Display booking history	Click "My Bookings"	All parcel bookings listed with status	Pass
TC-BH-3	View invoice	Click "View Bill" on a booking	Invoice with QR code displayed	Pass

4.2.2 Test Cases for System Testing

System testing was carried out after the modules were integrated. End-to-end workflows covering parcel booking, staff recommendation and assignment, delivery updates, delivery confirmation, failed-attempt recording, rescheduling, payment, feedback, notifications, and ML training were tested against the implemented behavior.

Payment (eSewa / COD)

Table 4.5: Test Case for Payment (eSewa / COD)

Test ID	Description	Input	Expected Output	Result
TC-PY-1	Pay via eSewa	Select eSewa, redirect to gateway	Signature verified; status 'Completed'	Pass
TC-PY-2	Pay via COD	Select Cash on Delivery	Booking confirmed; payment 'Pending'	Pass
TC-PY-3	Cancel eSewa payment	Cancel on gateway page	Redirect to failure page; status unchanged	Pass

Staff Dashboard

Table 4.6: Test Case for Staff Dashboard

Test ID	Description	Input	Expected Output	Result
TC-SD-1	Login as staff	Staff email and password	Redirect to Staff Dashboard	Pass
TC-SD-2	View assigned parcels	Click "Assigned Parcels"	Assigned parcels list displayed	Pass
TC-SD-3	Update delivery status	Change status dropdown	Parcel status updated successfully	Pass

TC-SD-4	Confirm delivery	Enter receiver name & remarks	Parcel marked Delivered	Pass
TC-SD-5	Record failed attempt	Select reason, add remarks	Status set to ‘Delivery Attempt Failed’	Pass
TC-SD-6	View delivery history	Click “Delivery History”	Delivered parcels history displayed	Pass
TC-SD-7	Record failed attempt and reschedule	Reason + next delivery date	Status set to 'Rescheduled' and attempt added to timeline	Pass

Admin Panel

Table 4.7: Test Case for Admin Panel

Test ID	Description	Input	Expected Output	Result
TC-AP-1	Login as admin	Admin email and password	Redirect to Admin Dashboard	Pass
TC-AP-2	Manage parcels	Assign / reassign parcel to staff	Parcel assigned; assignment logged	Pass
TC-AP-3	Manage staff	Add / update / delete staff	Staff record updated successfully	Pass
TC-AP-4	View analytics	Open Analytics page	Charts and KPIs render correctly	Pass
TC-AP-5	View activity log	Open Activity Log page	Chronological audit entries displayed	Pass

ML Delivery Prediction

Table 4.8: Test Case for ML Delivery Prediction

Test ID	Description	Input	Expected Output	Result
TC-ML-1	Train with < 15 delivered parcels	Trigger training early	Training refused; reason shown	Pass
TC-ML-2	Train with ≥ 15 delivered parcels	Trigger training via admin page	Model trained; MAE/RMSE/R^2 shown	Pass
TC-ML-3	Book parcel before training	Submit booking form	Fallback rule-based ETA shown, labeled	Pass
TC-ML-4	Book parcel after training	Submit booking form	ML-based ETA shown, labeled ‘ml’	Pass
TC-ML-5	View Delivery Accuracy Report	Open admin report page	Predicted vs. actual delivery time and difference shown	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Lesson Learnt / Outcome

The development of QuickShip provided practical experience in full-stack software development. Using React.js, Vite, Tailwind CSS, Node.js, Express, MySQL, Sequelize, and REST APIs helped develop a complete web-based courier workflow. The project also provided experience in authentication, role-based authorization, payment integration, email notifications, QR-based tracking, database relationships, and delivery-exception handling.

On the backend, implementing Node.js with the Express.js framework provided a solid understanding of middleware, RESTful API design, JWT-based authentication, and asynchronous programming. A major technical highlight was the integration of the eSewa payment gateway, which taught us how to handle secure third-party API communications, manage transaction states, and ensure data integrity between the application and an external financial service.

A major learning outcome was implementing the Random Forest delivery-time prediction module and evaluating it using k-fold cross-validation. The system refuses to train when fewer than 15 usable delivered-parcel records are available and uses a rule-based fallback instead. This helped distinguish the trained machine-learning model from deterministic calculations and rule-based recommendation logic.

Furthermore, working with MySQL and Sequelize enhanced our skills in relational database design, specifically in maintaining data consistency across complex relationships between users, parcels, deliveries, and payment records. Overall, this project improved our problem-solving capabilities, particularly in debugging full-stack issues, reasoning about small-data machine learning honestly, and managing version control using Git and GitHub.

5.2 Conclusion

QuickShip provides a centralized web-based courier management workflow connecting clients, staff, and administrators. The implemented system supports parcel booking,

transparent cost calculation, QR-based status tracking, staff recommendation and assignment, eSewa and COD payment, delivery confirmation, failed-attempt recording, rescheduling, feedback, analytics, activity logging, email notifications, and delivery-time prediction.

Clients can book parcels, receive a tracking ID and QR code, view the booking timeline, see a delivery-time estimate, pay through eSewa or choose COD, and provide feedback after delivery. Staff can update delivery status, confirm deliveries, and record failed attempts with an optional next delivery date. Administrators can manage users, parcels, assignments, analytics, feedback, activity logs, and the ML prediction module.

The Weighted Staff Recommendation algorithm provides a transparent scoring method using workload, distance, availability, and success rate. The Random Forest module is evaluated using k-fold cross-validation and the Delivery Accuracy Report compares predicted and actual delivery time. These features provide useful automation while the system remains explicit about its limitations.

The technology stack — React, Tailwind CSS, Node.js, Express, MySQL, Sequelize, and ml-random-forest — provides the foundation for the implemented courier workflow. The current version does not include live GPS tracking, turn-by-turn route optimization, SMS notifications, or email ownership verification for password reset. These limitations are documented rather than presented as implemented features.

5.3 Future Recommendations

The current version meets the implemented core requirements, while several enhancements could be added in future versions to support larger-scale operations:

- **Dedicated Mobile Application —** developing native Android and iOS applications using React Native would provide delivery staff with GPS-enabled tracking and push notifications, making on-the-field updates more convenient.
- **Live GPS Tracking & Route Optimization —** incorporating a mapping API for live staff location and turn-by-turn route suggestions would reduce fuel costs and improve delivery speed, replacing the current “last known delivery destination” approximation.

- **Richer ML Feature Set —** as the delivered-parcel dataset grows, additional features such as time-of-day, day-of-week, weather, and staff-specific historical speed could be added to the Random Forest model, along with periodic automatic retraining and drift monitoring.
- **Advanced Notification System —** future iterations could integrate automated SMS alerts alongside the existing email notifications, to provide instant updates on booking confirmations, payment receipts, and delivery milestones.
- **Multi-Gateway Payment Support —** to provide more flexibility to users, the payment module could be expanded to include other popular Nepalese gateways such as Khalti, IME Pay, and Connect IPS.
- **Deeper Analytics —** extending the existing analytics dashboard with peak-delivery-time analysis, staff performance leaderboards, and revenue-growth trends would give administrators more data-driven decision-making tools.

REFERENCES

[1] T. Aravind, P. Dileep Kumar, and D. Saketh, “Automated Courier Management System,” Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol. (IJSRCSEIT), vol. 5, no. 2, pp. 420-423, Mar.-Apr. 2019. DOI: 10.32628/CSEIT195290.

[2] M. Deshmukh, M. Kaslekar, L. Mathankar, Y. Qureshi, and P. Singh Sisodiya, “A Comprehensive Study on Courier Tracking Application,” Int. J. Technol. Appl. Sci. (IJTAS), vol. 16, no. 11, 2025.

[3] E. Rabuan, J. Zainudin, H. Samad, F. Miserom, and S. Sabri, “Courier Tracking Management System with Notification Using Barcode Scanner,” Asian J. Prof. & Bus. Stud., vol. 1, no. 1, Jun. 2020. DOI: 10.61688/ajpbs.v1i1.11.

[4] N. Durai, N. M., and S. V., “Automatic Courier Management System,” Int. Res. J. Eng. Technol. (IRJET), vol. 6, no. 3, Mar. 2019.

[5] H. S. Patel, A. D. Patel, and D. A. Patel, “Design and Implementation of Go-Courier: A Centralized Courier Management System,” Int. J. Sci. Res. Sci. Technol. (IJSRST), vol. 12, no. 4, pp. 212-216, Jul.-Aug. 2025. DOI: 10.32628/IJSRST251293.

[6] C. U. Betrand, C. G. Onukwugha, C. I. Ofoegbu, and O. B. Aliche, “Logistics Web Application for the Tracking of Parcels,” Int. J. Data Sci. Technol., vol. 10, no. 1, pp. 11-17, Feb. 2024. DOI: 10.11648/j.ijdst.20241001.12.

[7] D. K. Hiuredhy, H. J. Christanto, C. Dewi, and S. A. Sutresno, “Optimizing Logistics: Developing an Efficient Tracking and Delivery System for Automotive Spare Parts,” Jurnal JTIK, vol. 8, no. 4, pp. 1060-1070, 2024. DOI: 10.35870/jtik.v8i4.2380.

[8] E. O. Asika, “Web Based Courier Management System,” ResearchGate, May 2023.

[9] S. Ammulu and K. M. S. Reddy, “Online Courier Management System,” Int. J. Innov. Res. Technol. (IJIRT), vol. 4, no. 11, pp. 1387-1391, 2026.

[10] Y. Jaiswal, S. Dubey, and H. Gupta, “AI Based Courier Tracking and Automation,” Int. J. Prog. Res. Eng. Manag. Sci. (IJPREMS), vol. 6, no. 1, 2026. DOI: 10.56726/IRJMETS51235.

[11] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5-32, 2001. DOI: 10.1023/A:1010933404324.

[12] R. Kohavi, “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” Proc. 14th Int. Joint Conf. on Artificial Intelligence (IJCAI), vol. 2, pp. 1137-1143, 1995.

APPENDICES

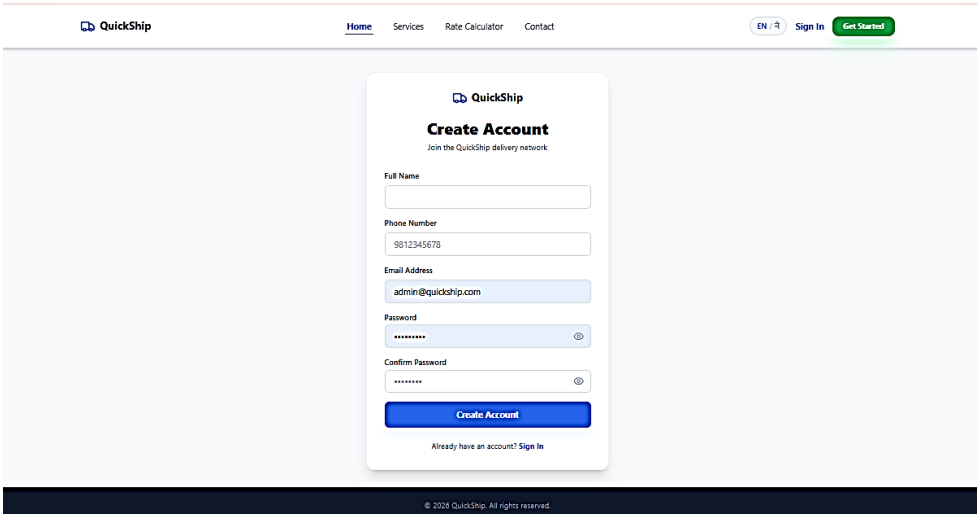


Figure A-1: Registration Page

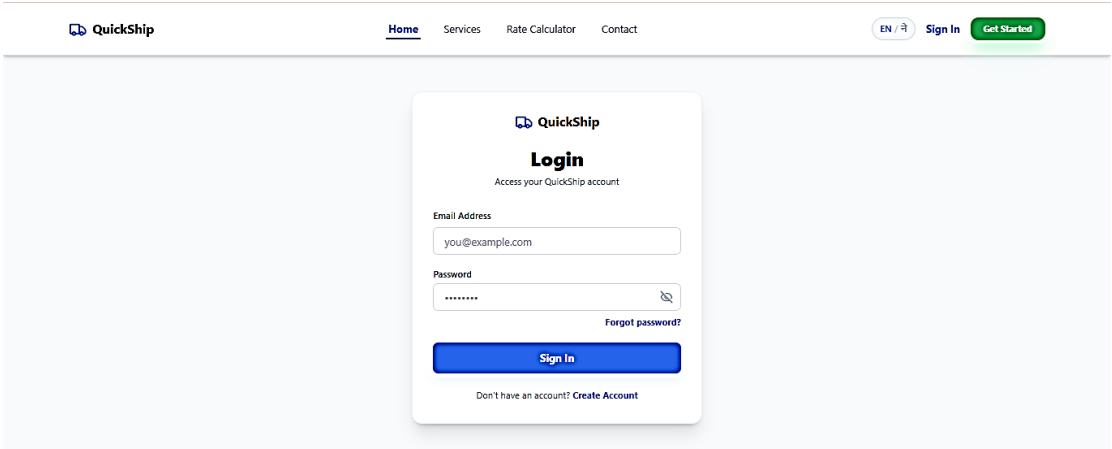


Figure A-2: Login Page

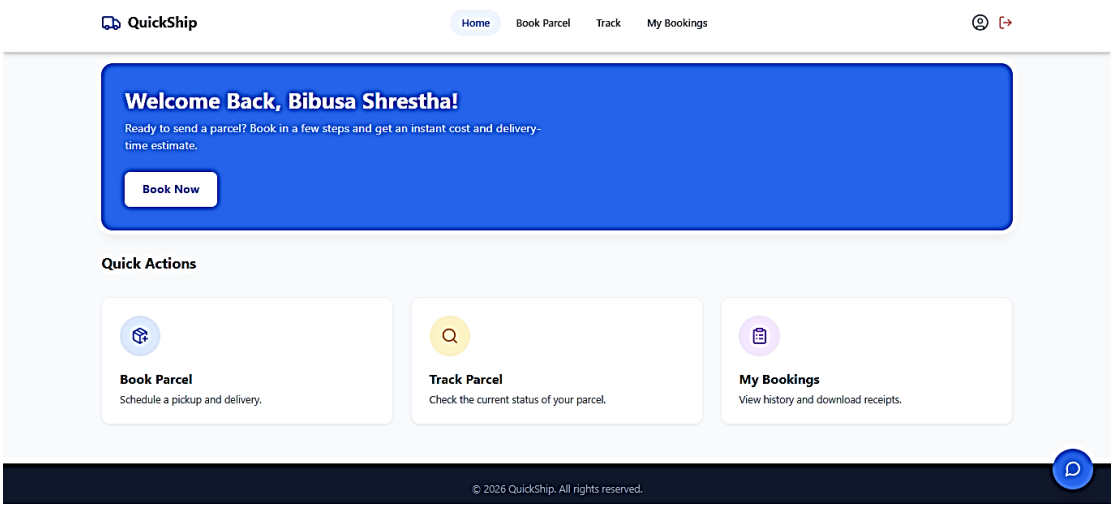


Figure A-3: Client Dashboard

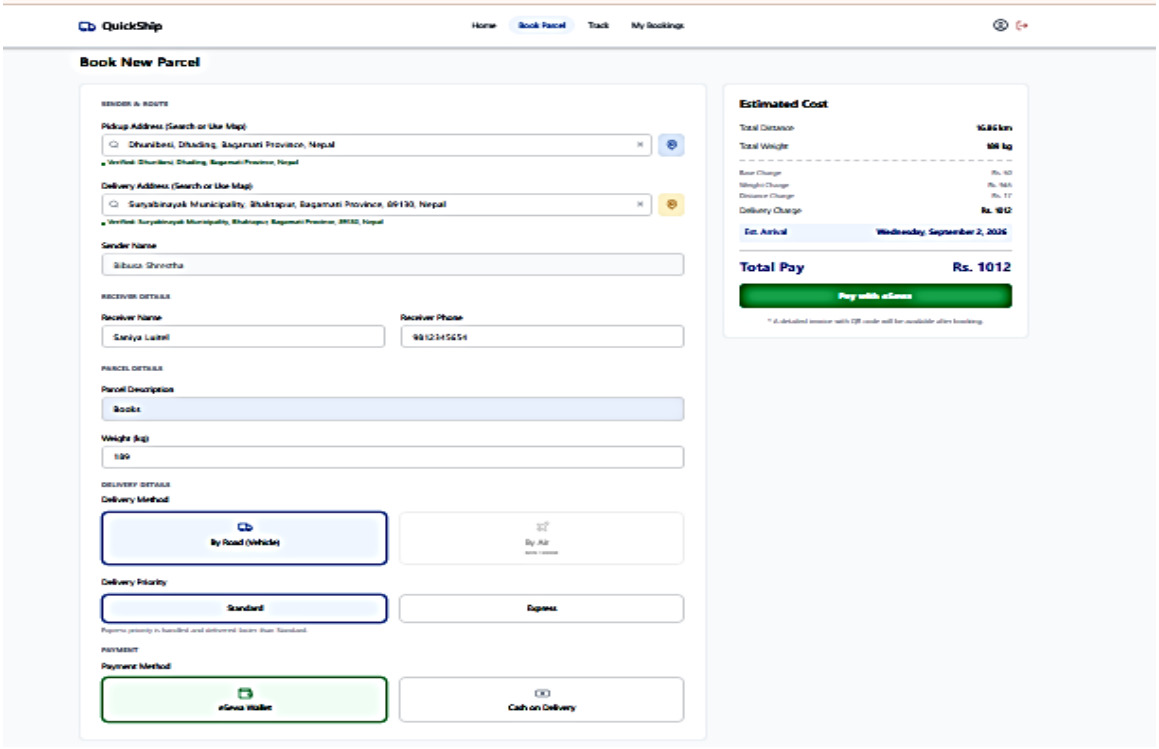


Figure A-4: Parcel Booking Page

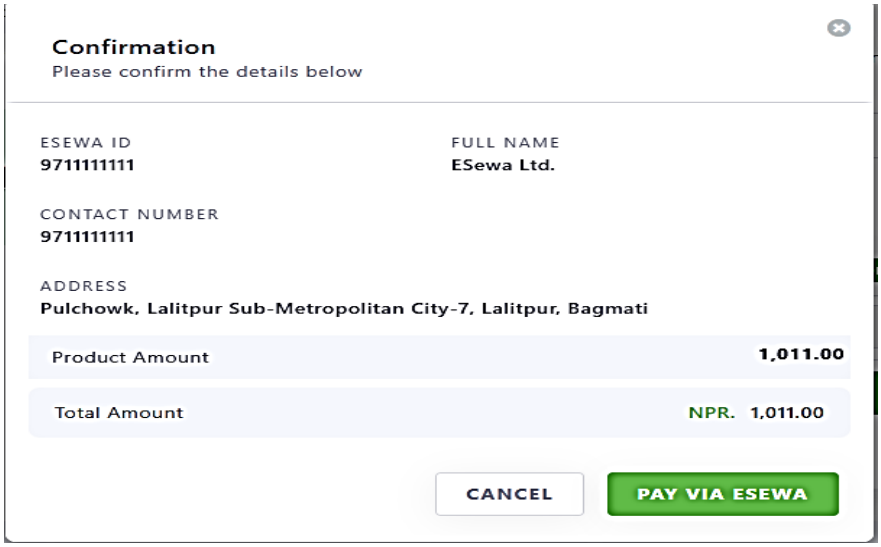


Figure A-5: Payment Confirmation Page

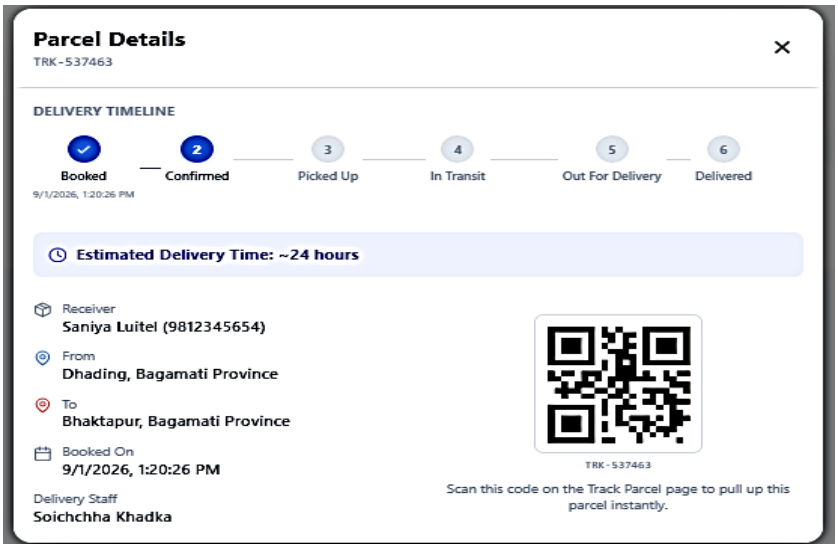


Figure A-6: Parcel Details

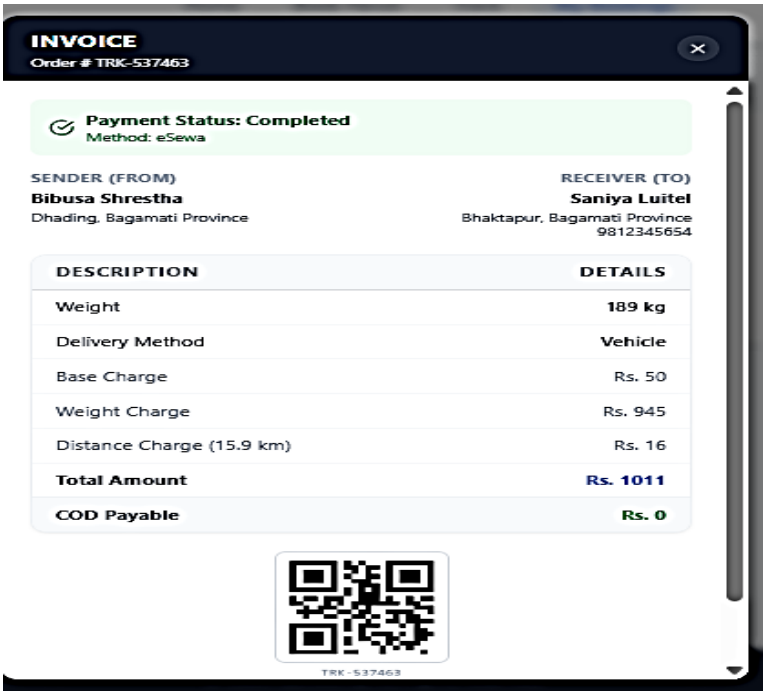


Figure A-7: Courier Bill

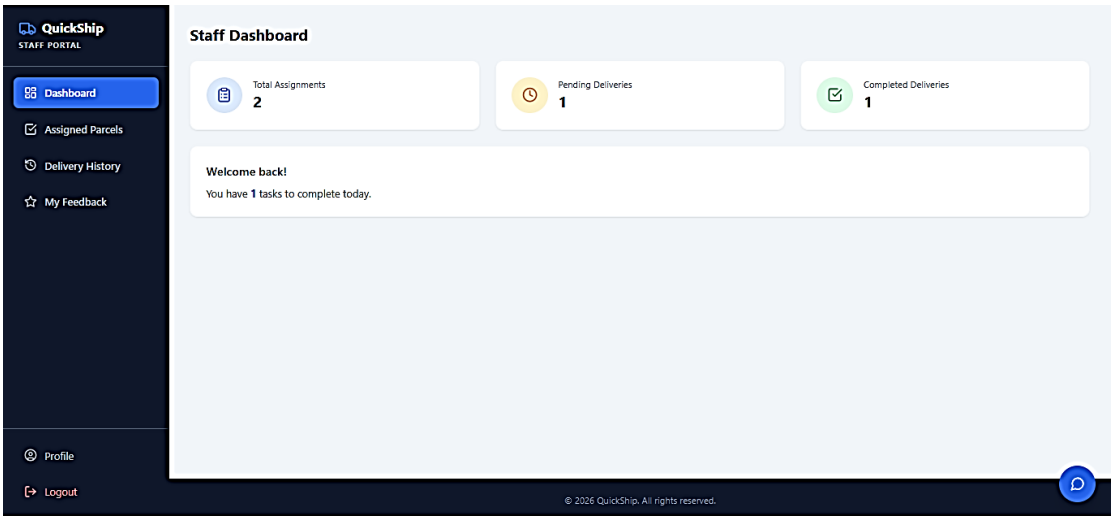


Figure A-8: Staff Dashboard

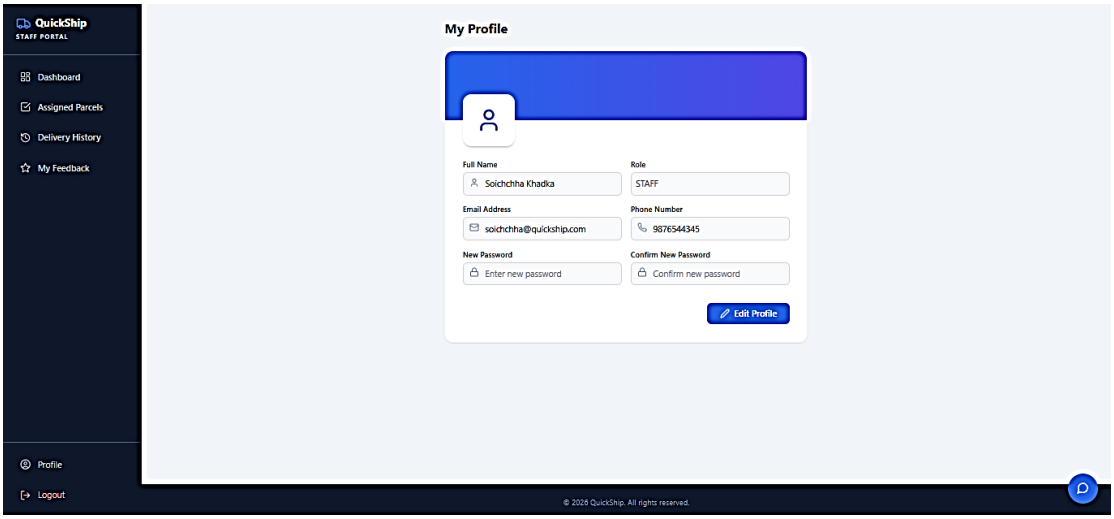


Figure A-9: Staff Profile Page

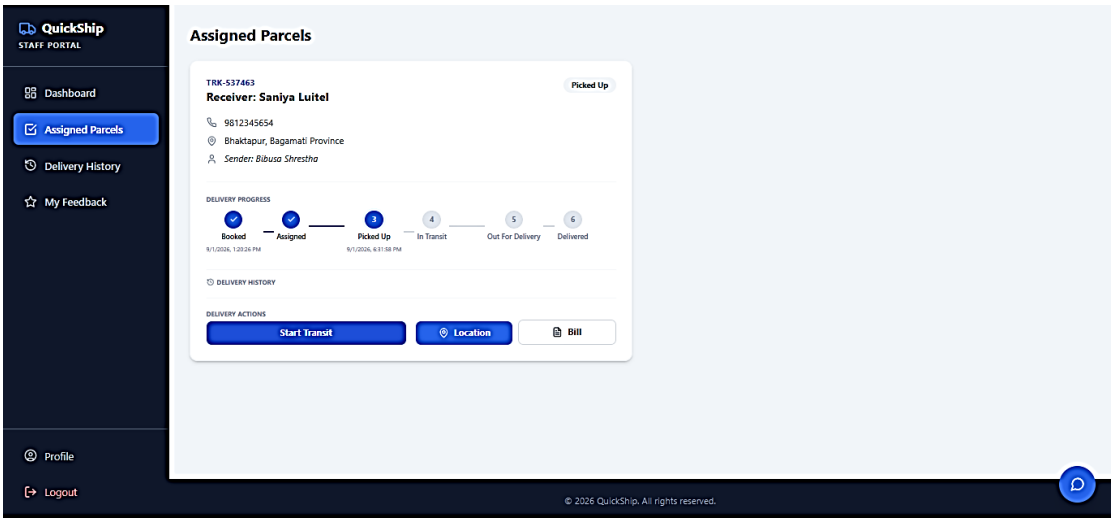


Figure A-10: Assigned Parcel Page

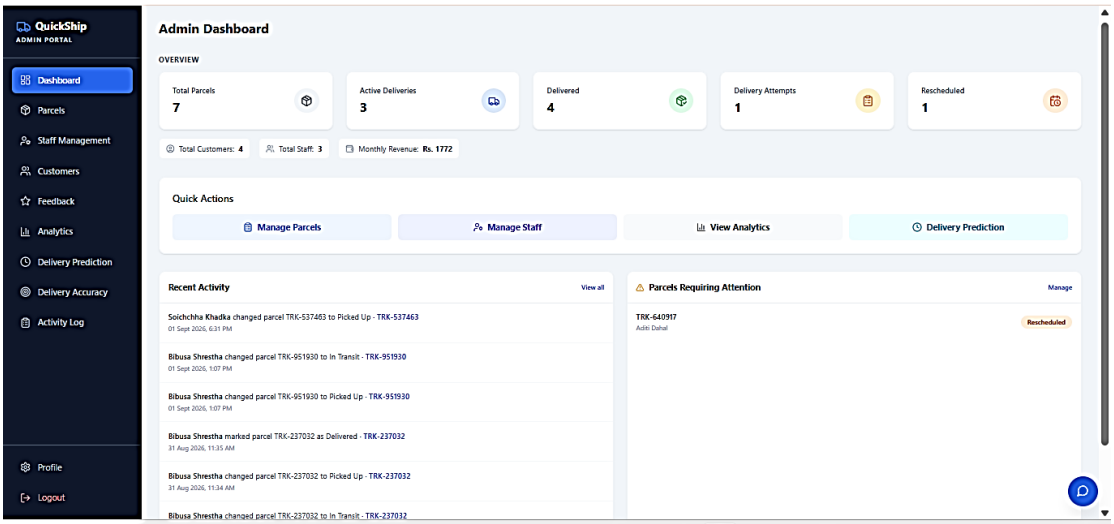


Figure A-11: Admin Dashboard

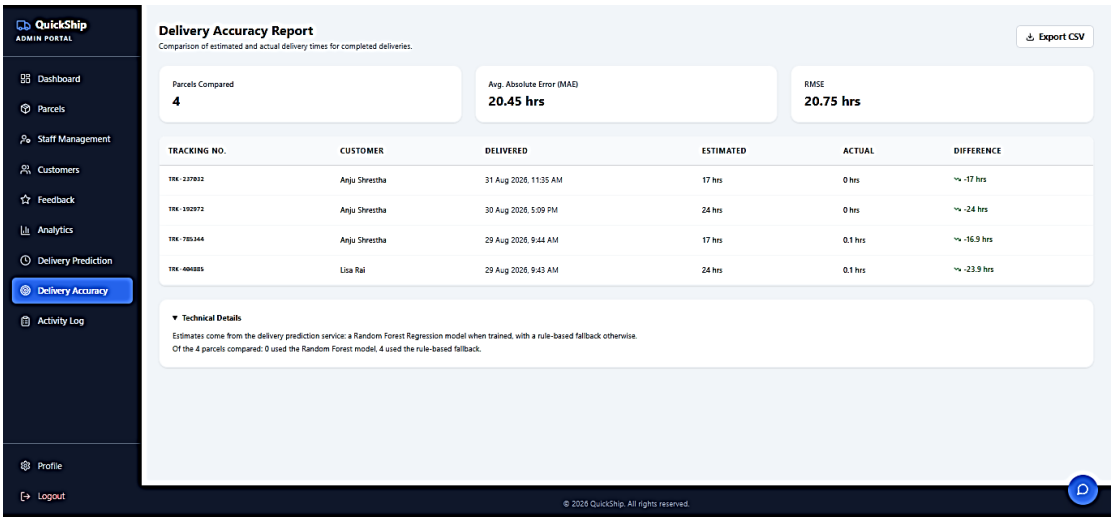


Figure A-12: Delivery Accuracy Report

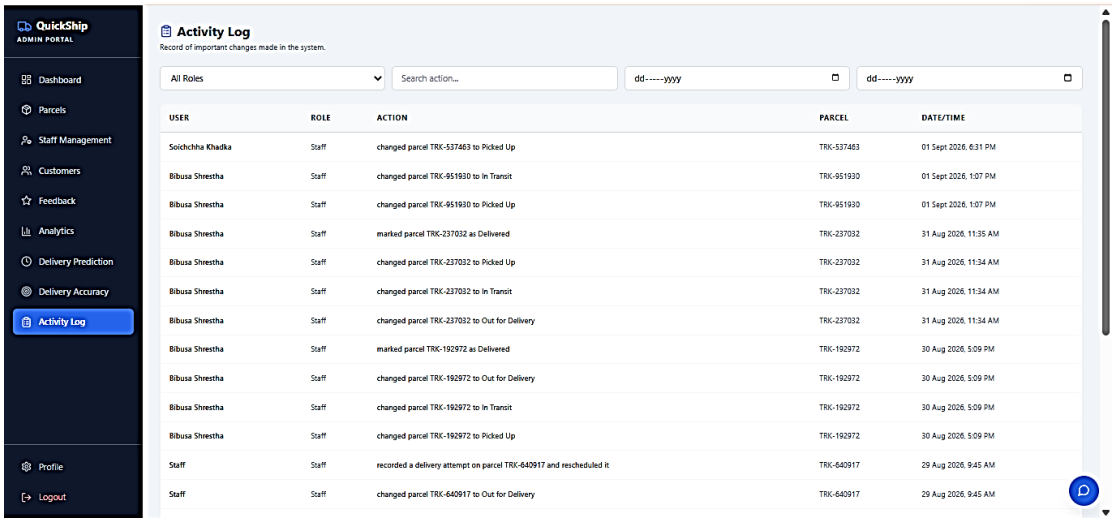


Figure A-13: Activity Log

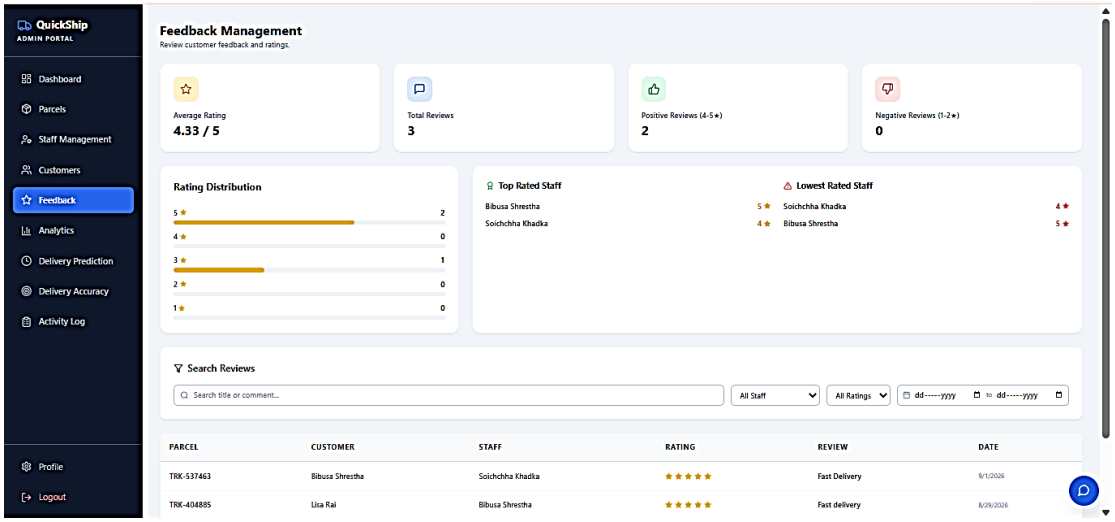


Figure A-14: Feedback Management

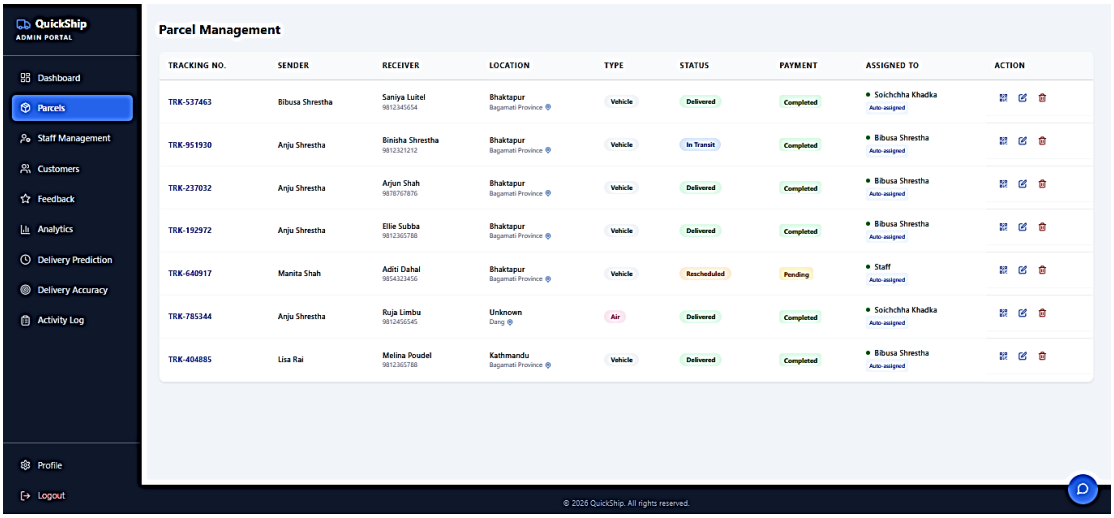


Figure A-15: Parcel Management

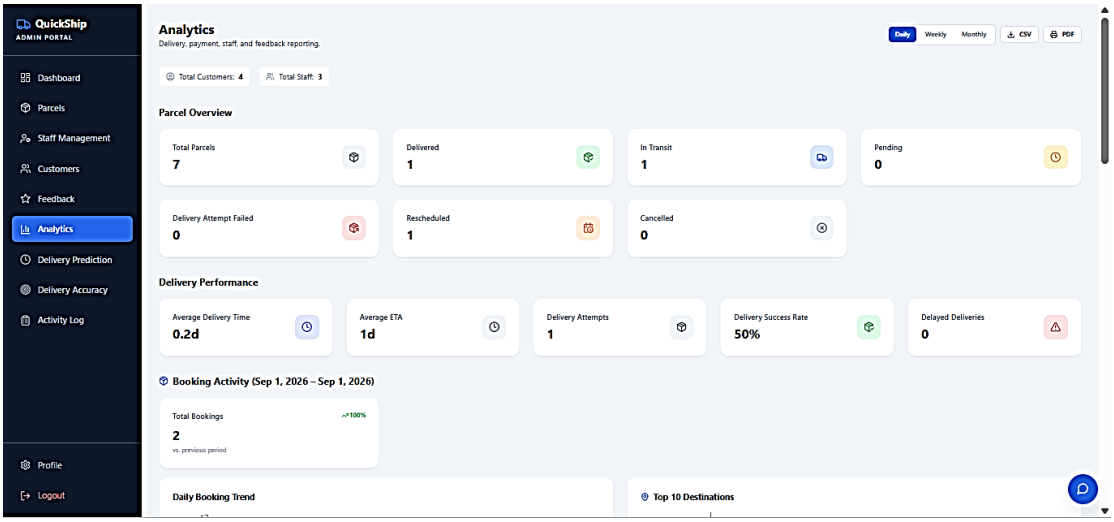
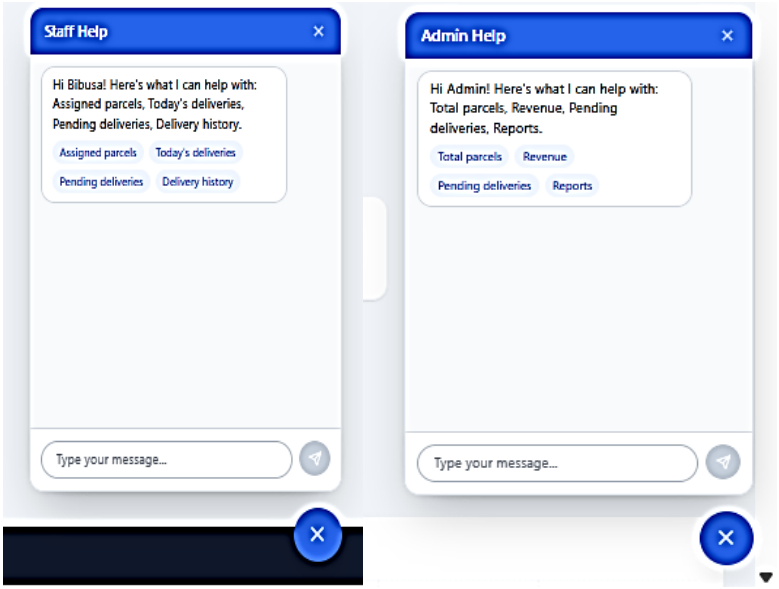


Figure A-16: Analytics Dashboard



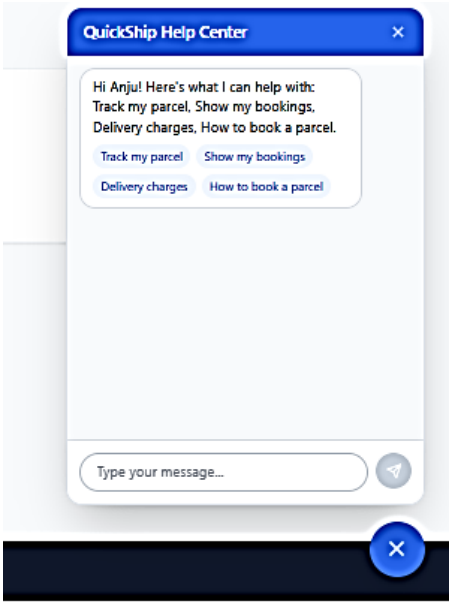


Figure A-17: Virtual Assistant